

3:17 / 3:19

Presenter: Let's talk a little bit about the different types of storage and connection modes in Power BI. In this course, all of the data sources that we'll connect to will use import storage mode but it's really important to understand that Power BI actually supports several different types of storage and connection modes and that they're each useful in different types of scenarios. Import mode is the default storage mode where tables are stored in memory within Power BI and queries are fulfilled by cached data. Import data sets are really useful when you need fast query performance, and the dataset will be less than one gigabyte after compression. These are also super handy when the source data doesn't change frequently and when you don't need restrictions on Power Query, the data modeling, and then DAX functions. Direct query tables are connected directly to the source data and any queries, meaning data requests, that are needed are pushed back to the source to be executed on demand. This storage method is most commonly used when data sets are too large to be stored in memory and the source data updates very frequently, and reports must reflect the most recent data. Additionally, this is commonly used when a company policy states that data can't be imported

but can only be accessed
from the original source.
Composite models, as you may
have already guessed by the name,
are models that contain a mixture
of both import and direct query modes
or are created from multiple direct query tables.
Composite models are useful in cases where you
want to really boost your data model performance
by setting appropriate storage types
for each table or combine a direct query model
with additional imported data,
or create a single model
from two or more direct query models.
And our last option here,
live connections are used to connect
to pre-published Power BI data sets
from sources like Power BI Service
and Azure Analysis services.
Live connections are great
because they create one data set that serves
as a central source of truth.
All team members can author
and create multiple different reports
from the same underlying source,
and it also helps multi developer teams work
more efficiently and seamlessly,
where one team member may build the model
and another team member works on the visualization.
As I mentioned at the start of this lecture,
we're gonna be importing all of our data sets
into Power BI's memory,
and we'll be using import mode.
From a little bit of a higher level,
part of your role as an analyst
or a Power BI developer
may be to recommend data connection strategies
based on the business case or the business need.
The point here is that

I don't wanna cover all of the intricacies and nuances of each of these connection modes, but rather, I really just wanna help make you aware that there are different ways that you can connect to data within Power BI, and that the reasons for using those are going to vary. So each of these storage modes definitely has different types of features and restrictions, and limitations and upsides and considerations, and it's really important to read more about each type of these storage modes prior to using it. All right, so with that, we're gonna jump into our next lecture.

Instructor: So far, we've seen the process for connecting to and loading data from a local file into Power BI, but chances are you won't always just be working with flat files. So in this lecture, I'd like to cover the process of connecting to a database. Now, Power Query can connect to data from many different database sources like SQL Server, Microsoft Access, MySQL, Postgres, Oracle, SAP, and many, many more. And the great thing is that once you understand how to connect to one database, the process is very similar to connect to other database sources. From the Get Data window, you'll select the database connection you want and enter the database's specific credentials. Once you've added in the appropriate credentials, there's some advanced options that allow you to configure the command timeout and even write custom SQL statements to customize, limit,

and shape the data before it's added into the query editor.

Once these details are all set,
we'll be able to select the specific tables
and then load them into the query editor
for QA, shaping, and transformation.

So for this demo,

I'm going to be using a database that we use
in Maven's advanced MySQL course
called Maven Fuzzy Factory.

And if you've taken the MySQL course,
you should have the community server
installed on your machine
and can likely follow along with this demo
and practice the connection steps.

Just to call this out, specifically,
we will not be installing Community Server
as part of this demo.

So if you don't have it installed,
no worries, just sit back, relax, and enjoy the demo.

So with that, let's jump on over to Power BI
and we'll check this out.

All right, so from the relationship view here,
we can come up to Get data
and we'll click on more sources here.

And we can actually just type in mysql to search,
and we'll connect to our MySQL database.

All right, so to get started here,
we're gonna enter in our server and database name.

And this is my local server address.

And the database name is mavenfuzzyfactory,
and mine is called _development.

All right, and like we saw in the slides here,
we have some advanced options.

We can set a command timeout.

Or here you could type in some custom SQL statements
to either limit, or transform,
or shape the data prior to importing it.

But we're just gonna connect to the database as is.
So click OK.

And at this step, when we're prompted for the credentials,
I actually want to use the database credentials.
So I have a username and a password set up.
And my username is root, and I'll enter my password.
And we'll connect from here.
Awesome, so we have our kind of preview window here, right?
Looks similar-ish to when we connect to the CSV, right?
Except for now, we have our different table names.
And if we click on each one of these table names,
we're gonna see a preview of result
on the right-hand side, right?
So we've got all of these different tables
that we can click through and kind of see, right?
And we're gonna connect to all of these tables, right?
All six.
And again, we've got load and transform data options.
We're gonna click Transform Data,
and that'll bring us back into the query editor.
And now that we've got the data
all loaded into the query editor,
from here, we can go through
all of those same kind of applied steps.
We can update our table name,
we can go through and check all of our data types,
rename column headers,
go through really all of those different QA
and any sort of transformation steps that are needed.
But because we're not using this data throughout the course,
I actually don't need to perform
any sort of transformations here.
The one thing that I do want to do
is I want to disable this data from being loaded
into the data model, right?
I'd love for it to still stay in the query editor for now,
but I don't need it within my model.
So if I right-click this table,
I'm gonna click this Enable Load.
And there's a very subtle name change here
where the name is now italicized,

and that indicates that it is not going to be loaded, right?
So we'll go through each one of these
and remove the Enable Load.
And again, this is just a great tool to use
when you don't want to actually load those tables
into your data model.
All right, so from here, we're gonna click Save,
I'll apply my changes, and we'll be good to go.
All right, so that is your demo
on connecting to a SQL database.
All right, so now that everything has saved,
that's gonna wrap up this demo
on connecting to a SQL database.
Up next, we're gonna check out
how we can actually scrape data from the web.

3:55 / 3:57

Narrator: All right, I've got another quick demo for you.
And this time we're going to be using the Web connector
to scrape data from a Wikipedia page
and then import it into Power Query as a table.
Now, the really cool thing about this Web connector
is that it allows you to connect to web-hosted files,
or we'll scan a webpage, identify any structured tables,
and then provide a list of the tables found
and show you a preview
before loading it into the Query editor.
Now, it's super simple to use and really powerful.
Let's check it out.
All right, so for this demo,
we're going to be using the Wikipedia URL
that was added at the bottom left of that slide.
So feel free to pause the video, rewind a bit,
and then copy the URL so you can follow along.
All right, so what we're going to do here
is head up to a new source,

and Web is actually one of the common connectors here.

So we can just click Web.

So from here, it's as simple
as copying and pasting that URL.

If we were to click Advanced Options,
there's some other pieces here
where we can add different URL parts,
again, kind of command timeouts,
different HTTP requests, and parameters and stuff.
So we're not going to mess around with any of that.
We'll stick to Basic. We'll click OK.

All right, so what Power BI is doing here
is it's going through and connecting to that webpage,
and then it's going to load this very familiar
data preview window, right?

Kind of similar to the MySQL example that we saw.

We have all of the kind of different elements
or tables on the left hand side.

All right, and if we click one of these,
we're going to get a Preview window here.

The other view that you can see here,
that's actually really cool,
is you can actually click on this Web View,
and it actually shows you what the webpage looks like.

All right, so you can scroll through and, you know,
let's say we want to import this table
with rank, firm, company, country.

All right, we can see that this largest company's table
that we've selected, rank, firm, company, country,
is that table.

All right, if we started clicking
through some of these other tables, you know,
we're basically exploring the different table elements
that have been discovered within that page.

So if any of these other ones make sense to connect to,
you certainly could.

For the sake of this example,
we're going to connect to this table.

All right, so now that the data is in the Query editor,

we can go through and follow the same process that we've been using for our other tables.

All right, we'll go through and we can check that our data types and our column headers are appropriate.

All right, so we've got a whole number here.

Text makes sense for firm and company, and also for country.

And then for this, AUM column, right, assets under management, this is actually in billions of dollars.

So we could update this to currency or leave this as a whole number here.

And there are some other transformation steps that you could apply here to show the actual value here and not have it shortened.

So again, everything looks good there.

You know, we could update this name to Largest Asset Management Firms, and then we'd be good to go.

All right, so that is how you can use the Web connector to scrape data from a website and import any sort of tables that are visible within that page.

The last step that we'll do here is similar to our Fuzzy Factory data.

We're going to disable the load.

And finally, we'll save our work.

All right, up next we're going to dig into some of the data QA and profiling tools within the Query editor.

11:27 / 11:30

Instructor: Data profiling tools provide a visual way for you to explore your data and to get a sense of its composition, and each of the data profiling tools that we'll cover have some slightly different options, layout and purpose. Digging into column quality first, this is all about showing the percentage of values within a column and these percentages are broken down into three types, valid, error or empty, and if you're ever trying to quickly identify the percentage of empty or error values within a column this is a great tool to use. Additionally, you can hover over the column quality box to reveal a contextual menu with some details and additional options. On the lower right, you'll see a little ellipses icon that when clicked will reveal a bigger contextual menu with some other options to do things like cleanup, duplicates, remove empty values and remove or replace errors. So one thing to keep in mind here is that the great thing about these types of contextual menus is that they allow you to solve column quality issues without having to try and find the appropriate steps within the query editor. These menus really bring this functionality straight to you without you having to go and find it. Another profiling tool is column distribution and this provides a sample distribution of the data contained within a column and very similar to column quality tools, we have the same types of contextual menu options that help us clean up and sort out error or duplicate values that may be contained within the column.

The other thing that I'd like to call out here is that you actually have a suggestion from the query editor and this suggestion is a potential remedy based on the distribution that it finds within the column.

So here we can see that there's a removed duplicate suggestion to help clean up this data.

This may not always be exactly what you want, but it can be kind of handy if the recommended option is the exact data cleaning step that you're looking for.

Our last data profiling tool is called column profile, and this provides a more holistic view of the data within a column by providing a sample distribution of the data along with specific column statistics.

So on the bottom left of this view, we have the column statistics view, which provides a bit more detail than just the column distribution view we just reviewed.

This actually now gives us some detail about counts, the number of distinct values, the number of uniques, some min and max and empty values and so forth and so on.

On the right hand side, we have the value distribution which shows the distribution of each value within the column, and similar to the other profiling tools you can also hover to reveal a contextual menu with suggested transformation options.

All right, so with all of this in mind, let's go head over to the query editor and we're gonna check out these column profiling tools for ourself.

All right, so for this demo we're gonna connect to the customer lookup table, and this is the table that we're gonna use to demo some of these profiling and QA tools.

All right, perfect.

So again, kind of following some of our best practices and initial first steps, let's update our table name here to just customer lookup, and then we'll check to make sure

that all of the data types and column headers look good.
All right, so kind of scrolling along here.
Text values, our birthdate as a date, marital status
gender, text, email, text, annual income here.
This is a whole number.
We could update that to currency if we want,
but that looks good.
Okay, everything looks really good here.
So all of these column profiling tools
that we talked about are located up here in the view menu
and let's enable our column quality,
and we can see that our quality, you know
it actually looks pretty decent, right?
We get this little results window for valid error and empty.
Like I said, when we hover, we get these contextual menus.
You can right click and also get a contextual menu
with some of these kind of
common transformation steps, right?
So if we scroll along here, right?
We've got prefix column here
with less than 1% of empty values here,
and again like this is stuff
that kind of makes sense, right?
We may not have a Mr or Mrs. prefix
for all of the customers within our data set.
So seeing that it's only nine here,
I'm not super concerned about this,
and it's probably not some sort of issue
that is gonna compromise the integrity of our data, right?
If we keep scrolling along here, again, everything is valid
at a 100%, no errors, no empties
everything looks pretty good here,
but this is actually a bit misleading,
because the default column profiling only analyzes
the first 1000 rows of the table,
and our customer lookup table contains over 18,000 records.
So in order to view all of the records,
we need to update the column profiling
from the first 1000 records

to be based on the entire data set
and then once we update that range
we'll see how this really looks.
So to do that, we see down at the bottom here
where it says column profiling is based
on the top 1000 rows,
and we actually want to have it
based on the entire data set.
So once we do that, power BI goes through and re-scans this
and once this updates, look what happens, right?
Our customer key column now has a bunch
of errors in it, right?
We're seeing that there's five errors here.
Some of our other columns like prefix and first name,
like we're actually seeing some empty values
in here as well.
So something is going on with this data set
that we actually need to clean up,
and like we had talked about
we can actually use Power Query's suggestions here
to remove these errors, right?
So if I click remove errors here,
a new applied step is going to be added for that
and then we'll see the updated results based on that change.
The only other piece that we have here now
is that we're actually getting some empty values
within this column, right?
So again, we can follow Power Query suggested update here,
click remove empty, right?
We've added another applied step here for filtering rows.
We're filtering out those empty rows
and now we're kind of back to where we were at, right?
We've got 130 empty values here for prefix,
but again, that may make sense based on the data set
and if I scroll over, right?
All of these other columns are now a 100% valid, right?
So it looks like that took care of it,
but what I would still like to understand is
what exactly was causing those errors, right?

Like what was the actual values or the reason behind it?

So let's delete these last two applied steps, right?

And now we're back to the state

where we have all of these errors.

If we come back to our header here

and we right click, if you select keep errors,

what this is going to do is add a new applied step

that shows a results table

with all of those errors in it,

and you can actually click into each of these errors

to see the Power Query error message.

So this is a data format error

that we couldn't convert to a number, right?

And here's the details.

This 30 with three dashes is what Power Query

was trying to convert to a number, right?

And if I clear out of this last applied step,

you know, I can click into another error here,

and again it's that same error.

We couldn't convert this to a number

and here's the number that was trying to be converted.

All right, we'll check one more here.

All right, so this is same error,

but a slightly different issue, right?

It looks like there's some source data here

or a link or something like that,

that couldn't be converted to a whole number.

So again, like that is the underlying reason why

those errors were happening.

Those are the actual issues within that data set.

We also see that there's some other values within prefix

that really don't make sense, right?

These are not picked up as inaccurate,

because of the column data type being text,

but lowercase M doesn't really mean anything, right?

If we scroll over here,

looks like we've got some birth dates from,

you know January 1st, 1900.

So again, these different rows here don't make sense

to keep as part of this data set.
So if we clear our keep errors step,
the great thing about this
is that we can just rerun through
those same cleaning steps, right?
We can either remove errors by clicking this,
we can right click and then select remove errors, right?
We can right click again, remove empty
and after that we're back to this nice clean data set,
where our only empty values are
within this customer prefix column.
All right, so now that we've got these errors cleaned up
let's quickly check out some
of the other profiling tool options,
and I'm gonna deselect column quality.
We'll check out column distribution,
and again, when we select our column distribution here
we see a sample distribution
of the values within each of the columns,
and if we go in, let's check out first name here,
we can see that we have 666 distinct values
and 89 unique values.
So this means that there are 666 distinct first names
within the column, and 89 of them appear only once, right?
So that's how to interpret those numbers,
and if we hover over the column header here,
again Power Query has this removed duplicate suggestion,
and again, we're a little bit smarter
than this suggestion and understand
that duplicates are actually necessary within the column,
so we'll ignore that
and then last, we'll check out our column profiling tools
and here we get a really nice view of our column statistics
and a chart showing the distribution.
Again, one of the important pieces to note here
is that some of these column statistics
really may not make sense.
Count makes sense.
We're gonna wanna see the errors

or empty values, distinct, unique, these may make sense, but the min and max here, these don't really tell you much beyond the first and last names in an alphabetical order, but if we scroll over to a column like our annual income and select this one, now some of these column statistics are gonna provide a little bit more value here, especially when we get down to the minimum, maximum values our average and standard deviation, right? So depending on the data contained within each of these columns, some of the statistics tools and the distribution may make more sense. The larger point here is that you have a bunch of tools at your disposal that are really helpful to use when you're exploring and trying to understand the data contained within your tables. Data prep and QA is often a very time consuming and intensive process as an analyst, and these tools can help make that process a bit quicker and scalable.

Instructor: Next up, we've got tech specific tools.

And looking at the Transform tab and the query editor, you'll notice that Power BI groups different sets of tools together based on their purpose or function. So in this case, we'll find all of our tech specific tools grouped together at the end of the ribbon. And within this group we've got some really interesting and powerful options. So for one, we can split up a column based on a specific character or delimiter or based on a number of characters. We can format any of our text columns using basic formatting options, like lowercase, uppercase, proper case, which is capitalizing the first letter of each word.

Or we can use tools like Trim, which eliminate leading and trailing spaces, or clean, which does the same thing and also eliminates non-principal characters. Now you might be thinking that those trim and clean options really aren't that helpful, but once you run into a case where you have one trailing space in your data set, and trust me this will drive you crazy the first time you experience it. The problem is, as human beings we're incapable of seeing a trailing space. It's completely invisible to us and it looks exactly the same as any other data point without the trailing space. But to Power BI, it looks like a completely different and unique value. So in cases like that, trim or clean can be great tools just to standardize and help avoid issues like that, especially if you're working with really messy or unstructured text data. We also have some great extract tools. We can extract a certain subset of characters from a string based on a specific length. We can extract the first or last number of characters or a range. But where it gets really interesting is using delimiters, which we've already used as part of our first assignment. So what we can do here is we can actually tell Power BI that we want to return all of the characters before a specific delimiter or symbol or character or after it, or between two distinct delimiters. And there are also some advanced options as well that allow you to specify whether you search from the left side of the string or the right and if you want to skip a certain number of instances of each delimiter before returning the text. So just some really great flexibility there with those extract tools. So you may have noticed that some

of the tools here are grayed out or inactive,
like the merge columns or parse.
And that brings up a really important point,
which is that this toolbar is completely dynamic based
on what you've selected.
So if you've only selected a single column,
the merge column options are irrelevant,
so you can't even click them.
You'd have to select multiple columns
in order to activate that option.
And then taking that even further
if you've selected a column that's numerical instead
of text, this entire group of tools might be replaced
by numeric based or number specific tools instead.
So just remember that the entire ribbon, the entire toolbar
that you're seeing here will dynamically change based
on your selections.
Now, one more very important point to highlight
before we shift gears into Power BI,
anytime you see this yellow box that says,
hey, this is important,
I'm gonna talk about something really important.
So that means focus in and pay attention
because this is gonna be something that'll come
up time and time again throughout the course.
And it's really, really important
for you to fully grasp and understand.
So what I want to talk about now is the difference
between transform and add columns
because this is something that confuse me for a while
and it's really important to understand the difference
between the two.
And what I was noticing was that the same tools, in fact
the same identical sets
of tools kept popping up in different places.
And for the longest time, I just thought
that this was really confusing and redundant,
until I realized that the outcome is completely
different depending on where you select the tool.

So when you select a tool from within the Transform tab,
you're essentially modifying
or overriding the column that you've selected.
But when you choose a tool from the Add Column tab
you're creating a brand new column within your table.
So that may sound really obvious when I say it now,
but I guarantee as you're learning this tool
and as you're playing around with the query editor, you will
at some point in time select the tool from the wrong tab.
And you know what?
That's okay because obviously nothing is set in stone.
It's as simple as just deleting the last applied step
and you'll be back where you started.
But it is something to keep in mind
and hopefully it will help you at least understand
where you've gone wrong
and why these tools appear in multiple places.
So with that, let's head back to Power BI
and we're gonna get our hands dirty with some
of these text tools.
So continuing with the customer lookup table,
one of the things that's been bothering me
about this table is the prefix and the customer name
and first name columns.
So if I scroll back over here, right, everything's
capitalized and it just it looks pretty jarring
to me when I look at this.
I would like to see this in a proper case.
Let's first update our prefixed column, right?
So we'll come up here to add column,
and we're gonna come down to format.
And within the format menu option we have see our lowercase,
uppercase, and then capitalize each word.
This is the proper text option, right?
So we can click here and oh, so all right, so we've actually
added a new column to the end of our data set here
and it's called Capitalize Each Word,
which is a pretty bad title there.
But at any rate, what we've done here is instead

of selecting this from the Transform tab, we've actually selected the format option from the Add Column tab. So again, we're adding a brand new column to the end of the data set.

And again, just to highlight like this is what I'm talking about, it's really easy to just select the wrong menu item and create a brand new column instead of transforming a column or vice versa.

You could do it the other way around.

Again, the great news here is that it's easily fixed, right? We can just come in here, click X to delete that, apply to step, we'll come back over to our prefix column and now we can head up to our transform menu.

And then from here, head over to our text column tools, click format, capitalize each word, and now we're in the right spot.

We've transformed the values in this column to a proper case, or actually only capitalizing each word.

All right, so the next thing that I'd like to do is also apply that same transformation step to the first name and last name columns as well.

And so what I could do is I could select each one of these one at a time, come up to format or I can multi-select by holding shift and click, come here to format, capitalize each word.

And then the Query Editor creates those applied steps to change first name and last name both to proper.

Awesome, so we're in good shape there.

The other interesting piece here to call out is we didn't add a new applied step here for capitalize each word.

If we actually look at the M code here, we can see that we're transforming the columns to proper text, right? Text dot proper.

And what Power Query has done is it added two more conditions here for first name and last name after prefix.

Right, so we're just kind of adding or mashing that code together into one applied step

because the transformation is all the same there.
All right, so now that all
of these customer text attributes are updated
to proper case., what I would love to do
is create one brand new column for customer full name.
All right, and you can see here again,
we're in our transform tools.
So we want to head over to add column.
And what I want to do is I wanna select prefix, first name,
and last name.
So again, click on prefix hold shift
and select last name to select all three.
One thing to call out here is
that selection order matters when you're kind of merging
or combining things within Power Query, right?
So if you had selected last name first
and then prefix Power Query, interprets that
as you want the last name column first
and then first name in the middle, and then prefix last.
So just make sure you're selecting the columns
in the order that you want them combined.
So from here, and we're gonna come up to merge columns.
And again, here we're just setting our separator.
We're gonna add a space in between each one of these
we can create a new column name.
This is an optional step,
but again merged here is a pretty bad name.
So we'll call this full name, click okay and awesome.
You can see that we have our new insert merged column
applied step here.
Scroll to the end of the data set.
We've got our full name column where we have our prefix,
our first and last name.
And then the last thing that I want to do here
is just rename this applied step
to something a little bit more readable
so we can remember it.
Again, we don't have a lot going on right now
but this may change over time.

All right, so I'm gonna update merged to full name and we'll apply that change.

All right, so with those updates, I think we're pretty good with our customer lookup table here.

Last thing is let's head back to the Home tab and I wanna click close and apply to apply these changes and load them into our data model.

All right, so now that that's finished loading, can scroll over and you can see

that we have our new table here for customer lookup.

The other thing that's interesting and to take note of is that the tables that we have marked as disabled load are that we're not loading into Power BI data model, like our SQL Connections they're not showing up here, and that's expected.

So it's just great to confirm that and understand exactly what that functionality does.

All right, so I'm gonna save this and I will see you in the next lecture.

Instructor: All right, it's time
for your next assignment,
and you've got another email from your manager, Ethan.
"Hi!

We're looking to better understand
where our customers may be coming from,
based on their email domains.

Could you please create a new column
in the customer table that will allow us to do this?

Thanks! ETL."

For this assignment, you've got four key objectives.

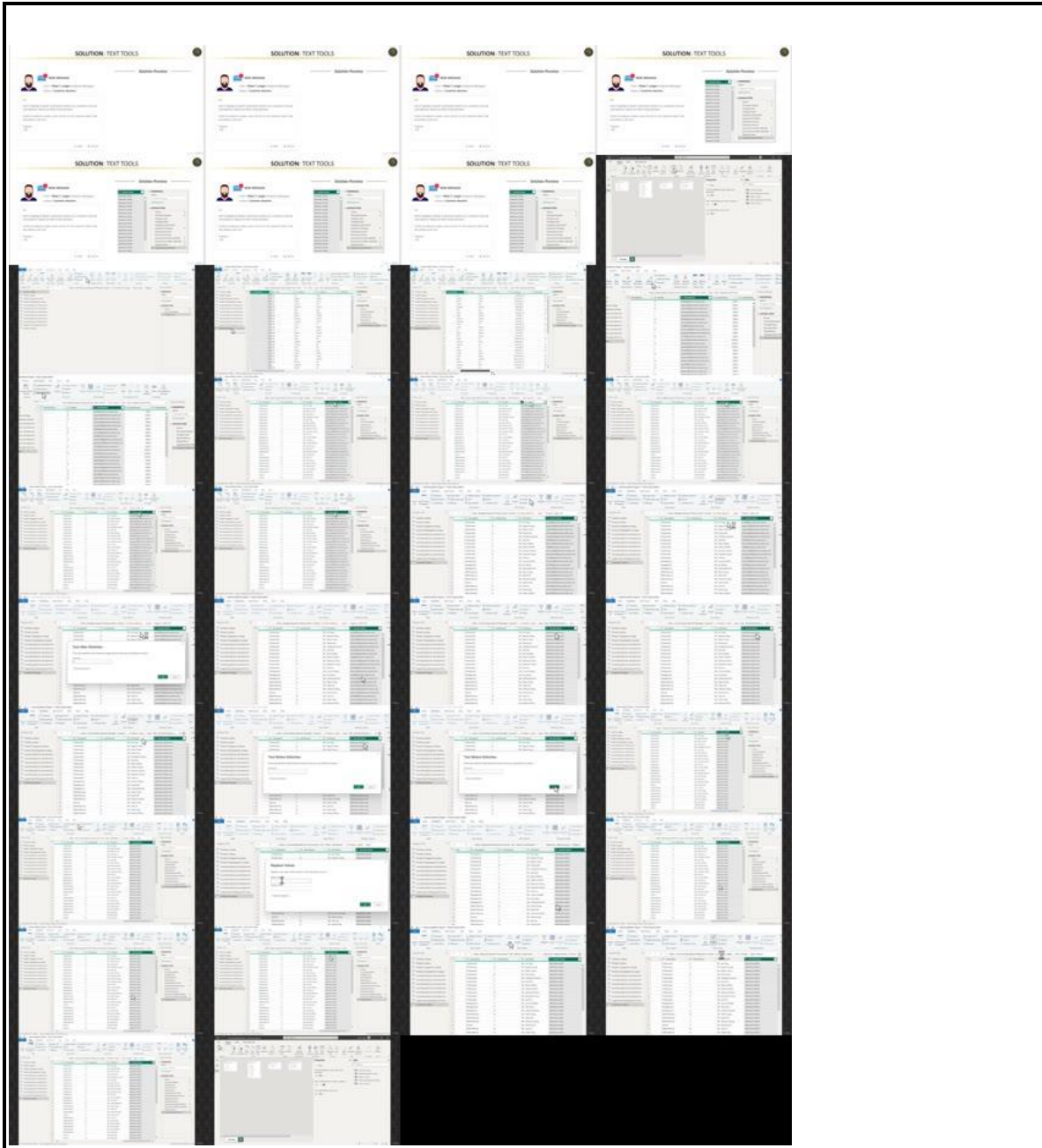
Number one, we need to duplicate the email address column
and name it Domain Name.

Next, in that new column,
remove all text or characters except for the domain name.

Next, we're gonna use transformation steps to clean up
and capitalize the domain name,
for example, Adventure Works.

And then last, we'll save and apply.

All right, have fun,
and I will see you in the solution walkthrough.



3:03 / 3:06

Instructor: Welcome to the

Assignment Solution Walkthrough.

As a quick reminder, Ethan had reached out

because they're looking to better understand

where customers are coming from based on their email domain.

Now, here's a solution preview of what you should see

in Power Query after you've completed the assignment.

Keep in mind that depending on how

you approached solving Ethan's requests,
your applied steps may look a little bit different
than mine, and that's totally cool.

The most important thing here is that you end up
with the same result.

All right, let's jump over to Power BI
and we'll check the solution out.

So to kick this off, we're gonna head to the query editor.

And once the query editor loads here,
we're gonna hop over into our customer lookup table.

And our first key objective was to duplicate
the email address column and then name it domain name.

So if we come over, select email address,
and then we want to head up to add column tools
and duplicate column.

All right, so we've added a duplicate column.

We need to update the name here to domain name.

All right, so that is our first key objective.

All right, our second key objective is in this new column,
we need to remove all text and characters
except for the domain name.

So if we come back over to our transform column menus here
and then we can use our extract tool options to extract text
after a delimiter.

So basically what we want to do here is say everything
that is after the at symbol we want to keep.

So we'll just click okay
and now we're left with this adventure-works.com.

The next thing that we need to do is I'm gonna tackle this
by removing the .com.

So again, from this extract menu,
I'm going to remove text before delimiter.

So basically we want to try to keep everything
before the period in .com.

So click okay.

And then the last piece here that I want to do is replace
this hyphen between Adventure and Works.

So if I come over to replace values,
and then the value to find is the hyphen,

replace it with a space, and then once this updates,
we should see Adventure Space Works.

All right, so our third objective here
is to use transformation steps to clean up
and capitalize the domain name.

So what I would like to do here is, again,
we're gonna use that proper function or the capitalize
each word here again, transform menu options, format,
capitalize each word, and we have Adventure Space Works.

All right, so back to the home tab, close and apply.
And that's our fourth key objective for this lesson.



7:45 / 9:49

Instructor: Up next, we have our number specific tools.
So let's get back to that transform tab in the query editor.
And assuming we have a numerical column selected,
you'll see a group of number specific tools that are active
that'll look something like this.
And the first option within those number tools
are the statistics functions.
And these are aggregators

like some, min, max, median, average, standard deviation, count, and count distinct.

Now, you may be wondering,
if you have a whole column selected
and you apply an aggregation function
that's designed to return a single value,
how does that work?

What happens?

And I'll show you exactly how this works
as soon as we jump into Power BI, but the answer is that
these all return one value, meaning a single value.

So the entire table gets replaced
with that one single value.

And as a result,
you don't really use these statistics functions
within the query editor as a means of transforming
or preparing a table, but rather as a way to explore it
and maybe get some information about it.

So things like the count of products,
the maximum costs, the medium age, questions like that.

Now at this point, you also might be thinking,
hey, can't we use the column profile tools for this?

And the answer is, absolutely.

The column statistics
and the column profile view are the exact same
as what will derive using these statistics functions.
And this lecture is really all about exploring the tools
and options that are available to you
within the query editor.

Plus, you'll never know when these statistics tools
may come in handy.

Now the next set of tools within this group,
the standard scientific and trigonometry tools,
these actually allow you to apply road level operations.

So you can apply the same standard operation like
addition, multiplication, division
or some more advanced calculations like log,
sign, tangent, et cetera.

And these are actually applied

to each value within the column.

So unlike the statistics options, these tools are applied to each individual row within the table.

In addition to those standard scientific and trig functions, you also have info functions.

And these basically allow you to identify binary flags, either true or false or one or zero, to mark each row in a column, whether it's odd, even, positive or negative.

So with that, let's open up Power BI and we'll practice some of these number tools.

All right, so similar to the tech specific tools lecture, we're not gonna connect to new data for this lecture, but instead use the product lookup table.

So we're gonna head back into the query editor, through that transform button,

and we're gonna practice the numerical tools here.

Once we're in the query editor, let's make sure we've got the product lookup table selected.

And if we jump up to the add column menu, here are my number specific tools.

And you'll notice that the statistics options are grayed out, right?

Because they're aggregators, they're gonna return a single value.

So it doesn't really make sense to add a new column that's derived by one of these stats functions.

But when we move to the transform tab, you can see that we do have these options available to us, right?

And these statistics options are active here.

And you'll see all of these different aggregation functions like sum, min, max, median, average, and so on.

Let's say for example, we wanted to understand the number of unique products captured in this lookup table.

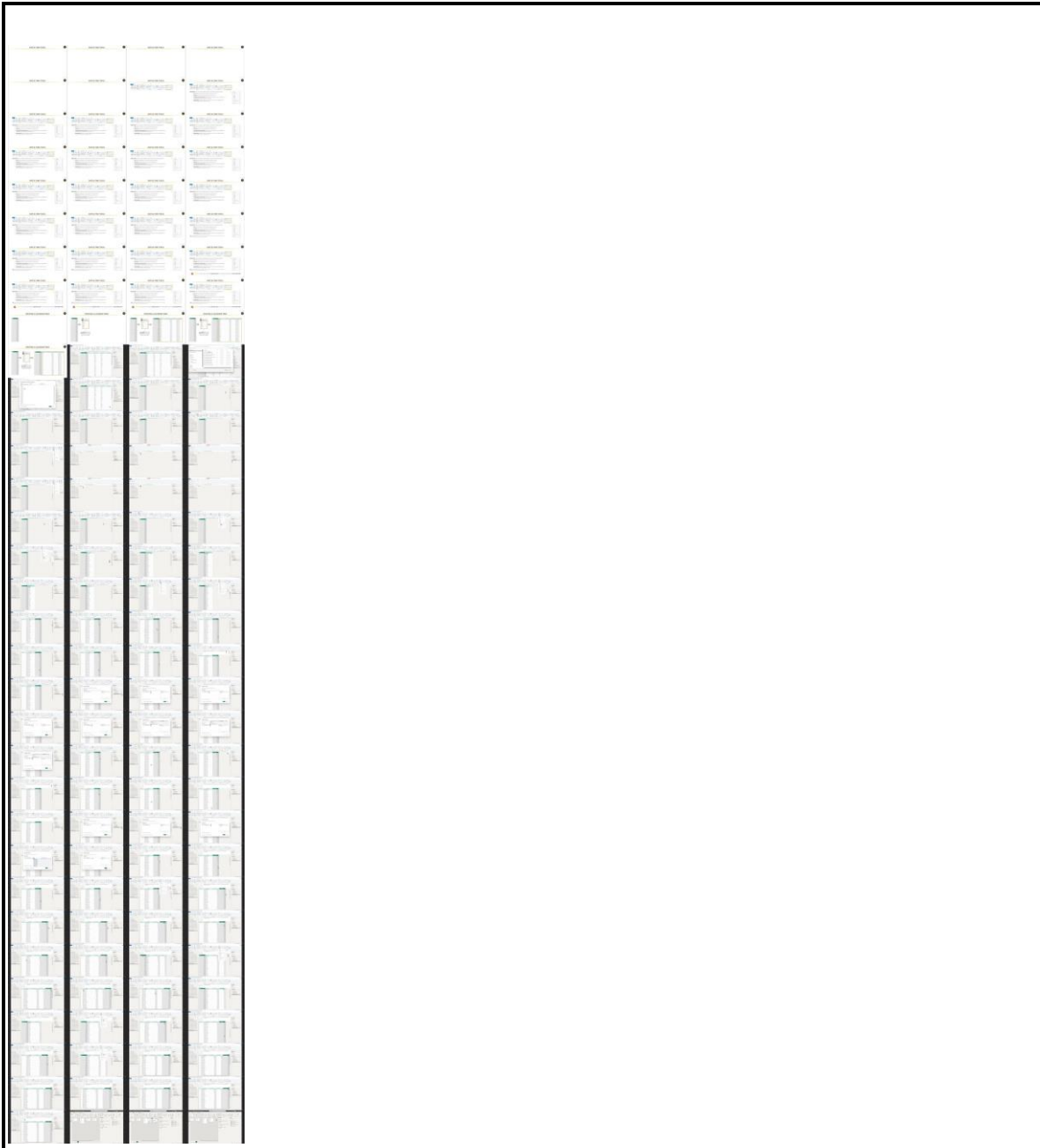
So if we click on product name and then head up to our statistics options, wait, we actually see here that a lot of these options are grayed out, right?

And that kind of makes sense
because we're asking about a text-based column, right?
And not a numeric column.
So you can't aggregate or sum texts
or find the minimum or maximum values
of a text value here, right?
All we can do is count or count distinct.
And in fact, all of my other number column tools
are actually grayed out here as well.
So let's head back here to statistics
and we're gonna count the distinct values here
of product name within the table.
And we have 293.
So what this tells me is that there are 293
unique product names in my product lookup table.
And like I had mentioned, as you can see it's pretty obvious
that this isn't a means of transforming or preparing data
to then load it into Power BI,
it's really better suited for exploratory analysis.
So almost every time you use these stats functions,
what you're gonna want to do
is delete this last applied step that's created
to return that full table.
All right, so let's look at another couple quick examples.
I'm gonna scroll all the way over to the right
and check out our product price column.
And now in this case,
I'm curious like, what's our average product price
in the Adventure Works data set?
All right, so I can click on product price here,
we'll go back to transform, and I can come back
to my statistics tools and click on average, right?
And we see here we get \$714 and 43 cents
and then a big remainder here.
This is kind of alarming, right?
It feels pretty high.
So if we close back out,
you can actually see here for our product names, right?
We've got these different road frames

you know, medium, large, all this stuff,
so remember this is a bike company,
and so we probably sell a lot of higher end bike equipment.
And you can see here in the product price column,
you've got some values here around 12, \$13,000.
Keep scrolling, we go up a little bit higher
to, you know, 3,500.
So now I'm curious,
can we use another statistics function
to answer this question a little bit more precisely, right?
What is our highest priced item within the dataset here?
So what we can do is, again, we'll go back
to this product price column, transform, statistics,
and let's find the maximum value, right?
And it's exactly what we just saw there.
So a little over \$3,500 here,
pretty expensive item here within the dataset,
and then we can click back on our X to clear this out.
All right, so it's pretty easy to use these tools
to explore some of the columns within your dataset.
So let's test out a couple more of these tools here.
And we're gonna scroll back over
to the product price and cost columns here.
And because we've already updated these data types
to a fixed decimal number, let's do a little test here.
Let's say we had left this as a decimal number, right?
We hadn't updated this change to currency step here.
All right, one of the things that we can do here
is we can use
the rounding tools from the transform menu
and we can either round up, down,
or we can specify the number of digits to round to
using that third option.
So let's say we wanna round this
to two decimal places, right?
And we generate this round off function.
And you can see here that we've rounded these off
to two significant decimal places,
but maybe that doesn't make sense for our use case,

maybe we want to keep the data type set to currency.
So let's delete that last applied step.
We'll delete the change type step there as well.
And now we're back here to our currency data type.
One of the last things that I want to show you here
is I want to do a demonstration of the standard operations.
So what I would like to do is not transform the column,
but we're gonna keep product price selected
and we want to add a new column
that's based off of product price, right?
And we're gonna do that
using one of these standard operators.
And what we're gonna do in this example
is we're gonna multiply our product price
by 0.09, all right?
I basically want to return 90% of the product price column
for each row within the table, right?
So we're gonna multiply this by 0.09.
And the way that you can think about this is,
let's say Adventure Works as a company ran a 10% off deal.
You know, this might be something like that
discounted product price column.
So we'll lock that in.
All right, so I can see here
that we have created the new column,
a new applied step here for inserted multiplication.
And we've got this, you know, pretty poor column name here.
So let's update this to discount price.
And the other cool thing to notice here is that because
this column is based off of the product price column,
it inherits the data type,
and when we multiply the product price by 0.9
our discounted price also is set as a currency
or a fixed decimal number data type.
All right, so I think that's just about everything
that we need to do as far as modifications
to the product lookup table.
So let's head back to our home tab
and we're gonna click close and apply.

And again, like we've seen,
this is basically just updating these queries
and it'll apply these transformations,
as you can see by what's happening on the screen.
And it's great.
It basically loads those tables, refreshes our data model.
So if I zoom in here
and I look at our product lookup table,
you can see at the top here,
we have our new discount price column
that we had created, right?
So that has been added into our data model.
All right, so there you have it.
That's your crash course in query editing number tools.



10:53 / 10:55

Instructor: Alright, so we're doing well here
and have already looked
at some tools that allow us
to perform data exploration in QA,
conduct some basic table transformations,
and manipulate tables with text and numerical tools,
and in this lecture,
I wanna talk about arguably one of the most important tables

in the data model, the date table,
and we're gonna check
out some date-specific tools within the Query Editor.
So, looking at the "Add Column" menu in our Query Editor,
you'll see those date
and time tools grouped here on the right,
and these are pretty straightforward.
You've got a few different options here,
and one of the more interesting ones right
at the top of the list here is the Age option,
and that actually automatically calculates the difference
between the current time based on today, or a Now function,
and the actual date in each row.
You have the "Date Only" tool,
which basically looks at any field that might contain
both a date and a timestamp
and it strips out the time component
and leaves you with just the date only.
You also have your pretty basic set
of date component functions like year,
month, quarter, week, and day,
and that just basically extracts individual components,
or individual attributes, from the date field,
and if you're selecting a field
that has a timestamp as well,
you'll also have similar time-specific options
like hour, minute, and second.
Two other ones that I wanna call
out here are Earliest and Latest,
and you'll see that those two are at the bottom
of the list and they're actually both grayed out,
and that's because we're in an "Add Column" menu,
and both of these functions act just like statistics tools
and they return a single value.
So, to access those two options,
you actually need to grab them from the Transform option,
and as you might have guessed,
the "Earliest" option is going to return
the earliest date in the column,

"Latest" is gonna return the last date.
So, a quick note here,
you'll almost always wanna perform these date operations
from the "Add Column" menu as opposed to the Transform menu
since most of the time,
the goal is to actually build
up new fields using these date components
as opposed to overriding
or transforming an individual date or time column.
So, pro tip here,
what you can actually do
in Power BI quite easily is load
up a table that contains a single set of dates,
or you could create a date table from scratch,
and then use these date tools to actually build
out an entire robust calendar.
So, that's exactly what I'm gonna show you how
to do in a second, but first,
let me give you a little sneak peek
of what it's gonna look like.
So, we're gonna start out
with a single column containing dates,
and then we're gonna apply some
of those predefined date options using
the "Add Column" tools
in the Query Editor,
and then we're gonna transform that individual column
into an entire calendar containing additional fields
like day of week, day of name,
start of week, month, quarter, year, et cetera.
Alright, so let's open
up Power BI and we'll give this a shot.
Alright, so if you're continuing along,
we had left off in the Query Editor here,
and we're gonna go
and open up our sixth CSV,
and we're gonna connect
to our AdventureWorks Calendar Table.
Alright, and the preview looks very straightforward.

We have just a single column containing dates.
So, let's click OK
and we'll add this into the Query Editor, and again,
let's keep our process consistent.
So, the first two steps that we're gonna take,
we're gonna update our table name to "Calendar Lookup,"
and then we're gonna confirm
that our column headers are updated
and that the data type is set.
So, we can see that we have date here
and that our data type is set as a date also.
Alright, so check and check. Everything looks great.
Now, let's explore some of these different date tools,
and I actually wanna start
with the "Earliest" and "Latest" options.
So, I'm gonna head up to the Transform menu,
and then I'm gonna come over to my date tools here,
and when I click "Earliest,"
it behaves just like those stats functions, right?
We transform the entire table
into a single aggregated value,
and we can see here that our calendar,
or our "Earliest" date starts on January 1st, 2020.
If I clear this out,
head back to my Transform tab,
and then we'll click "Latest,"
this is going to give us the latest date
in the calendar, right?
So, this is June 30th, 2022, alright?
So, we've got about two
and a half years worth of data within our calendar table.
So, let's close these steps out
and we'll get back to where we started.
Now, let's shift gears into actually adding columns, right?
Because that's what the real value is here, and this time,
I wanna create some new fields based
on this date column.
So, if I come up here to "Add Column,"
and let's say we wanna add in the day name first, right?

And again, you can see that these values are great out here because we're adding columns.

So, if I click on "Name of Day,"

we get a brand new column that's added.

We have an applied step here, insert a day of name,

and we see our column here, right?

"Day name" for the column header name

and then the data type is a text,

and we could update this if we wanted to,

but "Day Name" is actually pretty good,

so we're gonna leave this as is.

Another interesting piece here that I wanna show you is that the Query Editor actually has

some very interesting week-related calculations, alright?

So, if we head up here to Week, we can see week of year,

week of month, start of week, end of week,

and I wanna insert a "start of week" calculation,

and as we can see here,

as we look down when the start

of the week is actually occurring,

we can see that it happens on a Sunday, right?

January 5th is the start of the week, on this Sunday.

We skip down here,

we see the 12th, it's a Sunday,

so on 19th, the Sunday, so on and so forth, right?

But maybe that's fine, but maybe you wanna update it,

and to update it or customize the start of the week,

it's actually not super intuitive

and there are a couple of different ways that we

could actually accomplish this, right?

The first way is we could come up here to the Formula bar

and we could type in and edit the M code here,

or we can click this gear icon

and edit the applied step directly

from a window that pops up.

Alright, so now that we're in

this custom column window here, right,

this just makes it a little bit more manageable

to understand what exactly we're dealing with

instead of trying to edit the M code directly within the Formula bar, and this date function, "Date.StartOfWeek," has an optional second parameter, right? The first parameter is the date column that you wanna use. The second is the start, or the first day of the week, right? You can see it brought up here through the Intellisense option, and I know that if I type in "one," that returns Monday as the start of the week. So, we'll click OK, and you can see that our "Start of Week" column has shifted, alright? So, 1/6, it's a Monday, 1/13, it's a Monday, 1/20, it's a Monday, again, so on and so forth. You can also see up here in the Formula bar that the one has been added to this code. Say we wanted to change this to starting on a Tuesday, we could update that to two, and then we see the column shift again, 1/7 on a Tuesday, 1/14 on Tuesday, 1/21 on Tuesday, right? So, it's super easy to kind of update the start of the week. It's just not really intuitive at first. So, let's actually jump back in to the supplied step here, and while this is really straightforward, it's tough for people to kind of read this, right? Two or one isn't a very intuitive or readable format. So, our other option here is to come in and update this two to something a little bit more readable, like "Day.Monday, right? And that is going to show somebody, when they're looking at this code that, hey, the day of the week is starting on a Monday. We'll click OK, and again, we see that update back to Monday as the start date on the sixth, Monday on the 13th,

so this is great.

I want my week to start on a Monday,

so I'm gonna leave this alone.

Alright, so let's keep things rolling along here,
and I wanna add in some additional date columns, like,
let's add in start of month.

So, if I come back up to my date tools, click on Month,
and then click on "Start of Month,"

I can see here I've got this "Start of Month" column,
and on close inspection here, something seems off,
and what we need to be really careful

of is the column that we've selected
to base our calculated columns off of, right?

So, see how we were on the "Start of Week" column?

When we look at the "Start of Month,"

we're seeing the start of month is in December 1st,
December 1st, December 1st, December 1st, December 1st,
but our dates are actually in January, right?

It takes us until the 6th

of January to actually catch up and say, hey,
the start of our month is actually in January.

So, what we need to do is make sure we have
the Date column selected,

and then we can come over, go to Month,
"Start of Month," and here,

you can see the difference between the two, right?

Now, we're correctly assigning the appropriate start
of month based on the Date column instead
of based on the start of the week.

So again, this is something to really just keep an eye
on as you're working through adding columns.

If you have any sort of issues

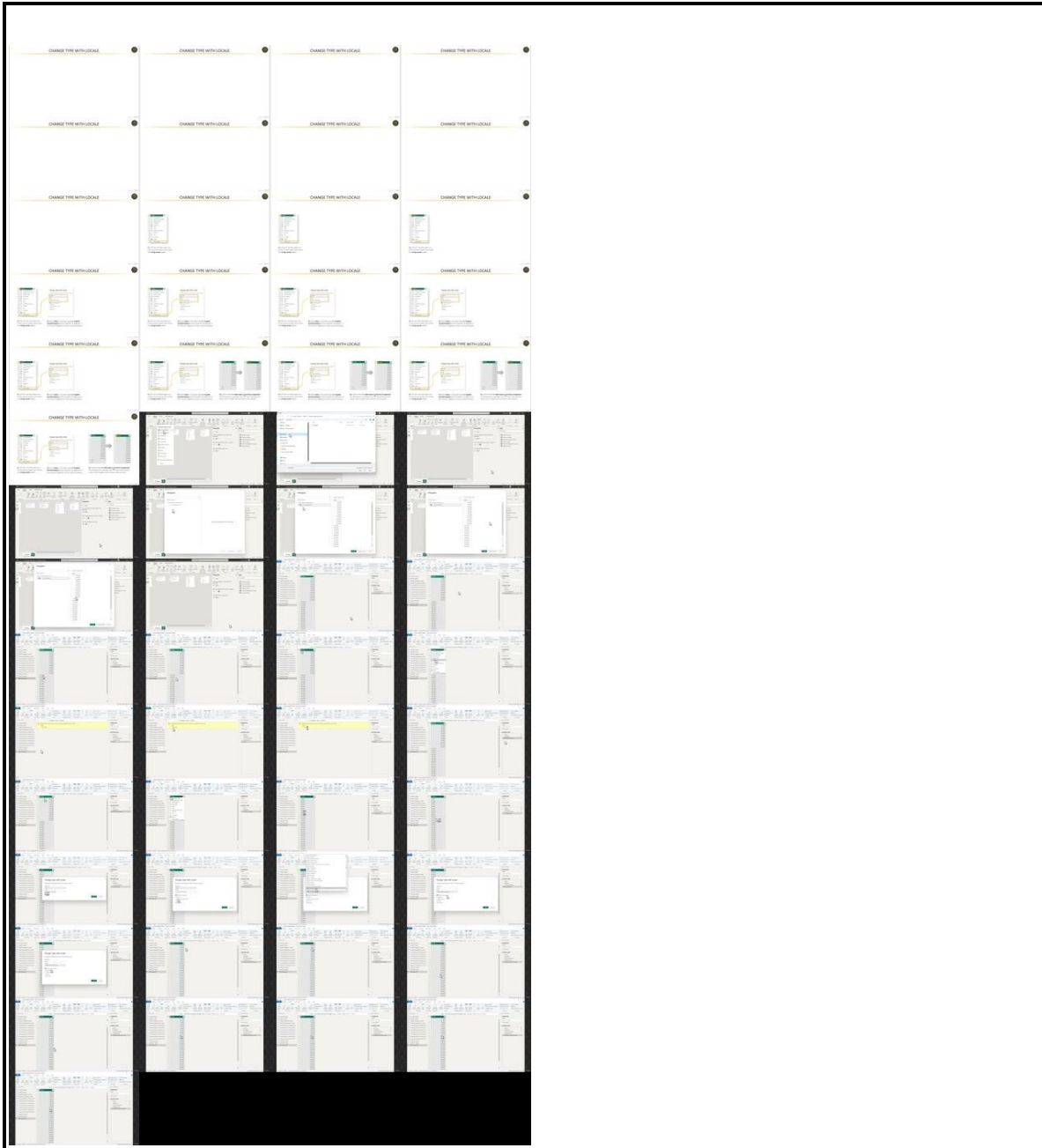
or errors within these calculated columns,
make sure that you're basing everything
off of the Date column.

So again, we cleared those applied steps out.

I'm gonna go back to my "Start of Month," and now,
we've got this appropriately set up, right?

All these starter months in January, we get into February,

and it's starting appropriately there as well.
Alright, so everything looks good now.
Let's add in one more column here
for our start of quarter,
and again, we had that Date column selected,
so our "Start of Quarter" values
are good to go there as well.
Alright, so this is a great start so far,
and I think you're getting the idea
of how quickly a calendar table can be built
from a single column of dates,
and what this calendar table will actually allow you to do
is to filter and segment the values
in a data set by any of these fields that we've created,
and this is gonna be really helpful when we start using more
of the analysis and visualization tools within Power BI.
Alright, so for now, I think our calendar is good to go.
We'll head back home.
We'll click "Close and Apply" to save these changes.
There we go.
We've got our calendar lookup table
that we just imported today.
Alright, so there you have it.
Those are our query editing date tools.



4:39 / 4:43

Instructor: I want to talk about a common error that you may experience when importing tables that contain a date or value from countries that use different standards.

So, for example, I'm in the United States and the default date standard here is month, day, year, but in the UK for example, the standard is day, month, year. So, what would happen if I imported a table

into my Power BI data model that contained UK dates?

Well, I'll show you exactly what happens in a moment, but the short answer is that I'd get an error starting on the 13th of the month.

The good news is that there's an easy process that we can use to update the locale used to recognize the date format, and remedy the error.

All right, so our first step in the process here, we need to make sure that all of the data types are the same for the values within the column.

And then, we're gonna left click the data type icon in the header and select using locale.

After that, and this is the most important piece to get right, you need to select date as the data type and then select the location that the file originates from.

So, in this course, all of the data types are based in English, United States.

So, if you're running into this issue, select that option regardless of your actual location.

If you select your own location the records won't update accurately.

And then third, you really just want to confirm that the updates have been applied and that the data type is correctly recognized and without any error or issues.

All right, so I'm gonna jump into Power BI and show you what this looks like.

There isn't a data set to follow along with for this demo, so just sit back, watch and enjoy.

All right, so we're gonna connect to an Excel file here that's located on my desktop.

I go back to desktop and we have this date error demo.

And we're gonna load this into the query editor to take a look at there and make some of these transformation steps that we talked about.

And with Excel, because there's multiple sheets that are possible within an Excel document, you're first gonna select the sheet

that you want to connect to.

And we have our same kind of data preview window here.

And immediately off the bat we kind of see that something's going on here, right?

When we get from the 12th or the 13th, there's some sort of issue.

So, let's click transform data, and we're gonna work through the process that we outlined in the slides.

First things first here, I wanna kind of demonstrate the way to not update this so you can kind of see what happens here.

Obviously, we've got something going on.

We're not seeing errors here on the 13th of the month, but we do have our values split out to the left and to the right side.

From the data type here, we can see that it's mixed between text and integer values or whole numbers.

So, what if we clicked here and just clicked update date?

All right, now we get all of these errors here, right?

We can click into this error.

Power Query is saying we couldn't parse the input provided as a date value, saying 13/1/2023, right?

It doesn't recognize that as a date value.

So, if we close out here and then we'll also remove that change type step.

What we want to do first is

because this is actually a mixed data type in the column we have both text and whole numbers, we actually want to update this all to text.

We wanna make sure that we're working with the exact same data type here.

Then we can come back in, we'll update using locale.

And for the purpose of this demo, this file has come from a friend in the UK, right?

So, what we want to do is we want to update this data type to date and the locale that we want to base the transformation off of is English, United Kingdom, right?

And we can see a sample of the input values, right?

So, we have the day, the month, and then the year,

which aligns with the way that this file is set up.
So, we'll click okay, and now look what happens.
Because I'm based in the US
and I want my date set up as month, day, year,
I now have that appropriately configured
for all of the values within the column.
We've got January 19th, 2023, January 20th, 2023,
so on and so forth.
And from here you can continue
and just use the add column tools to build out
additional columns based on this date field.
So, if you ever run into a situation where your date decimal
or currency columns are returning errors,
I'd suggest using this process to update
the data types based on the locale of the source.

Instructor: I wanna share a pro tip with you guys
that I think is pretty cool
and it's creating a rolling calendar.
So in the case that we've just walked through,
we loaded up a discreet set of dates
because we only want to analyze the data
that we have on hand.
Now, in our particular case,
we don't need to change that calendar.
We won't need to pull in new data
or analyze new information over time, right?
Everything's fixed.
But what if you were in a situation
that you had to, what would you do?
Maybe you've built a weekly report that you wanna refresh
with data every day of the week
and share with your colleagues, or your boss,
or your client.
Rather than creating a fixed calendar
and updating it over time,
we're gonna use some M code to create a calendar
that will be based on the current point in time.
So it'll act as a rolling calendar
that evolves as time goes on.
So I'm gonna lay out all the steps here in the slides
and then we're gonna jump into Power BI
and I'll demo what it looks like.
So step one, this is gonna be a great time to test out
that blank query option that I showed you
a little bit earlier in this section.
So we'll create a new blank query,
and then in our formula bar,
we're gonna generate a starting date
by entering something that's called a literal.
And you're gonna write it exactly as it's shown here
with the hash mark, the word date,
and then the starting date, whatever you choose,
doesn't matter what the starting date is,
but you wanna make sure

that it's in this year, month, date format.
So in the example we're looking at here on the slide,
this would represent January 1st, 2020.
So from there, you're gonna click on the fx icon
which adds a new custom step,
and you're gonna enter the following M code.
Now, I know it looks like it's a lot,
and we're not really gonna dig in
and unpack how this formula is working.
We're just gonna use this almost like a recipe
to create our list of dates.
So once you've added that formula,
a list will populate in the query editor,
and then we're gonna convert that list into a table.
Once you've added that formula,
a list will populate within the query editor,
and we're gonna convert that list into a table,
and then we'll format that column as a date.
And then last but not least, if you want,
you can add some more calculated date columns
like we showed in our last example.
You can always add years, months, week,
whatever you really want or need
using those add column options.
But it's really as simple as that.
Let's go hop over into Power BI,
and we're gonna write out this formula,
and I'll show you exactly how it works.
All right, so we're back inside our AdventureWorks Report
and we want to come to New Source, get Blank Query,
and this opens up a brand new blank query for us.
So for the sake of consistency, let's update our table name,
and we'll call this Rolling Calendar.
We'll lock that in.
And then the next thing we wanna do
is come up to the formula bar here, right?
And our first step is to use the literal.
So this is going to =# or pound
and date and then open paren and 2023,1,1.

So I'm gonna start this at the beginning of 2023.
We'll hit Enter.
And when you press Enter,
basically this creates a single value for January 1st, 2023.
So again, not super helpful on its own yet,
but the next step is to click on the function button
and we're gonna add a new custom step.
And you can see here where it says source.
This is where I'm going to type in
that long M formula that I shared with you.
And I'm gonna type this out fairly quickly
but I'll kind of talk it out.
So feel free to pause the video as needed.
All right, so here we're going to start off with list.Dates
and I will expand the formula bar here.
All right, and the next piece that we want to add in here
is source, and then Number.From open paren,
DateTime.LocalNow.
And then we'll do open close paren
like the TODAY or NOW function in Excel.
And we'll do - Number.From(Source), right?
We'll come over, we'll add one more comma
and Shift + Enter to return down.
And our last piece here is we're gonna add
in a duration (1,0,0,0), right, end paren.
We've got our closing parentheses, right?
We'll click enter to lock this in, and awesome.
You can see that we now have a list of dates
that are generated here.
So kind of scrolling back up here
without diving into exactly what this M code is doing,
in a nutshell,
what we're asking it to do is to calculate the current date
and then compare that against the literal
that we just created, which is our starting date, right?
And then based off the difference between those two dates,
I want you to create a list of all of the individual values
where the duration is one day.
So it's basically saying,

give me a list at the daily granularity between those dates,
the literal and the current date based on today, right?

So when we press Enter,

we can see that we have created that whole list.

See that our start date is here on January 1st, 2023.

And if I scroll down to the bottom here, keep going,

I'm down to April 20th, 2023,

which is the date that I'm actually recording

this lecture right now.

So the cool thing here is that if you come back tomorrow

or next week, or in a month, and you refresh this query,

all of those new dates will be added

or appended to the end of this list.

So that's basically what's going on here with this query.

Pretty easy to write out.

One thing to keep in mind here

is that M is case sensitive, right?

So if you did `ld` here for `List.Dates`,

that would throw an error.

So if you're running into those types of errors as well

make sure that your syntax

along with your capitalization is also correct, right?

So the next thing that we need to do

is we have this List Tools,

contextual menu that's popped up,

and we want to click Convert to a Table.

So here from this window, convert to table, right?

There's no delimiter, and we shouldn't have any errors

so we'll press OK.

We can see that we convert to a table.

From here, we want to do our one final step

is update our column name.

Let's call this date.

And then we also want to update the data type, right?

And we can set that to a date.

So that looks great.

And what's cool here is this is ready to use

just like the other date tables that we've imported

as we can click here, add column, go back to date,

and we can start adding in calculated fields like year.

Go back, we could add start a quarter
and maybe the last thing we want to do
is add start a month, right?

So in a few minutes with a little custom M code,
we're well on our way to creating
a scalable rolling calendar
that is gonna update based on the current date.

Now, one thing to keep in mind here
is that for the purposes of this course,
we don't actually need this rolling calendar.

We're really gonna rely on the calendar lookup table
that we've created instead.

So feel free to keep this query or delete it.

All right?

So that's gonna wrap up your pro tip lecture
on creating Rolling calendars.

Instructor: Our manager, Ethan, is back,
and this time, it looks like he needs some new date fields.
He says, "We need to add a few new fields
to our calendar table
to help us analyze sales trending over time.
Could you please add the following columns
when you get a chance?
Thanks! ETL."

All right, so for this assignment,
you have four key objectives,
and you need to add the following columns
to the calendar table.
So we need to add a column for Month Name,
for example, January,
Month Number, example, 1,
Start of Year, example, 1/1/2020,
and then Year, example, 2020.
All right, so that's it for this assignment.
I will see you in the solution walkthrough next.

Instructor: Time for the calendar table

assignment solution walkthrough.

And just as a reminder,

Ethan is looking for some additional columns

to be added to the Date table,

and those columns should look something like this

as long as you followed the order

that we laid out in the assignment,

but let's head over to Power BI and we'll check this out.

All right, so from our Calendar table in the Query Editor,

I wanna make sure we have the Date column selected

and we're gonna first add a new column for month name.

Let's go over to Date, Month,

and then we have Name of Month.

Right, so this is adding a new column

on the far right side of our table.

See month name set as January.

Right, still with that date column selected.

The next column we wanna add is month number.

So come back to Month,

and then this is just clicking the month value here.

Right, and you can see that this defaults

to that month number.

Right, our third objective here,

come back, make sure we have that date column selected

and we want start of year,

so Year, Start of Year, and perfect.

And then last but not least,

we wanna add in a column just for year.

Come up to Year, and then Year.

There you have it.

We've got month name as January.

Month number with the individual integer here.

And we can see that we've got all of these down to 12,

Start of Year, and then Year.

All right, so that'll do it

for the assignment solution walkthrough.



10:10 / 10:13

Instructor: Let's talk about two more types of columns that you can add from within the Query Editor, index columns and conditional columns.

So, starting with index columns, you'll find the option right here in the Add Column menu. And basically, what an index column is, is it's a list of sequential values that uniquely identify each individual row of a table.

Typically, these start from zero or one
and they're most frequently used
to create unique keys or IDs
that can be used to form relationships with other tables.
And I know we've been kind of
hinting at this idea for quite a bit,
and it's something that's gonna be covered
in much more depth in the intro
to data modeling section of this course.
So, index columns, they're super simple.
Moving on to conditional columns,
these are a bit more interesting,
and you can find 'em right there
above the Index Column option in the Add Column tab.
And basically, these allow you to define new fields
based on logical rules or IF/THEN conditions.
So the example we're going to be walking through
is to create a new conditional column
that we're naming Quantity Type.
And the way that we're gonna define Quantity Type
is a function of the Order Quantity column.
So, we can tell Power BI for any row in our table
where the Order Quantity equals one,
I wanna set my Quantity Type equal to single item.
If my Order Quantity is greater than one,
I wanna set my Quantity Type to multiple items.
And then you always have this catchall value if false.
So, that's basically, the value
that you want your column to take
in the case that all of the rows
or line items above are false.
So, Power BI is gonna go through and cycle through
each of these rows from top to bottom.
And if none of the conditions are met,
then that otherwise statement is the value that's returned.
All right, so let's open up our Adventure Works report
and we'll see what this looks like in the Query Editor.
All right, so at this point in the course
we're getting pretty close

to having all of our tables loaded.
And in this lecture,
we're gonna start working with our sales data.
So, first thing that I want to do here is
this query view here is getting
a little bit cluttered for me.
So, I actually wanna group together
a bunch of these queries that we have disabled load for.
All right, so I'm gonna select all of these queries
by right click, Move To Group,
and then create a new group here.
I'm just gonna call these Demo Queries.
All right, we'll click OK.
And basically, what this does
is it creates a folder structure
within your query pane here.
And again, it just really cleans this up.
So, now we can see exactly
the live queries here that we're working with.
All of these other ones are still accessible
and we could manage these
and mess with them at a later date.
But again, this kind of cleans things up.
All right, so we're gonna make sure
that we have our Other Queries folders selected here
and then we're gonna connect to our sales data.
And for this example, let's use the 2022 sales data.
Our data preview looks great.
It's pretty simple here.
We've got order and stock dates,
order number, some key columns,
and then Order and Order Quantity.
So, we'll add this into our Query Editor.
The table is added in our Other Queries folder here.
And again, let's follow those kind of first two steps.
We'll check out these column headers,
Orderdate, Stockdate, Ordernumber.
We've got date, date, a text field.
Our key columns are set up as whole numbers,

which look great.

And then same thing with
our Order and Order Quantity values.

The next thing that I wanna do
is we will update our name to Sales Data 2022.

And one thing to mention here is I'm specifically
not adding in that lookup identifier here,
because this is a different type of table.

This is something called a fact table, not a lookup table.

And we're gonna explain exactly what that means
in the data modeling section of this course.

But to give you a quick teaser,
essentially, we're looking at order quantities
and order line items here

with keys that allow us to map those orders
to things like our territories, to customers,
to products, to our calendar table.

And see here, if I scroll over to dates,
we've got some different date fields
that we can use to tie to that calendar lookup table.

So, no need to really dive any deeper than that quite yet.

Just put this in the back of your head,
because it'll come into play
in the next section of the course.

So, at this point, why don't we go ahead
and practice adding an index column.

So we're gonna come up to our Add Column menu.

Index Column, and you can see
from here we have some options.

We can start from zero or one.

Maybe you've got some sort of crazy business use case
where you need to start from a different Custom Number.

Again, you can specify that,
and with a specific increment as well.

Generally, I like to start from one, and perfect.

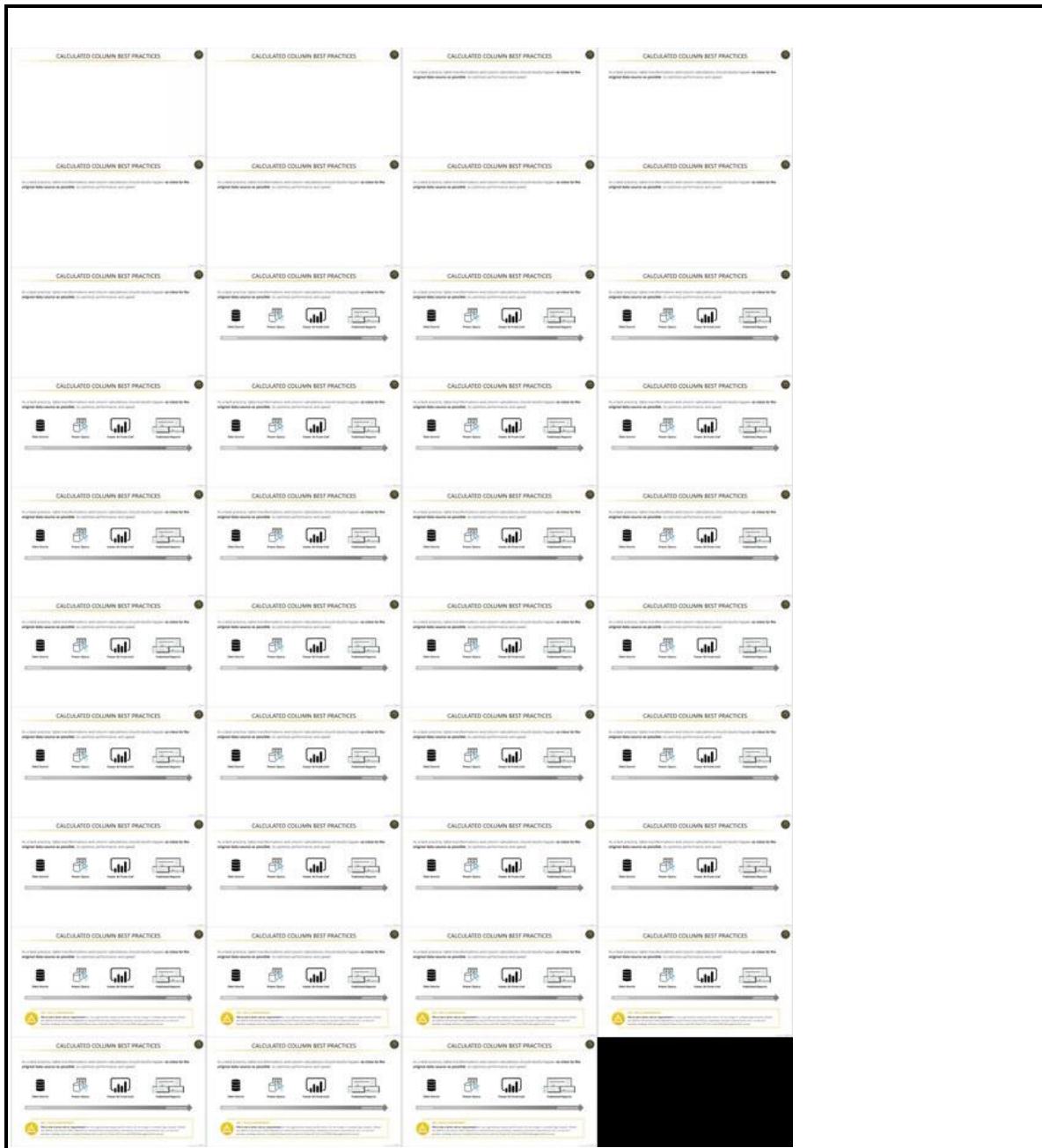
You see here, we've got a new applied step
for Add Index column.

So, let's say that we wanna see this
at the start of our dataset.

You can click a column header here, and you can drag it.
The other thing that you could do
to be a little bit more efficient
is you could right click, come down to Move,
and then move this column
to the beginning of the table.
So, now we've got our index column here,
we've reordered the column.
So we've got another applied step there as well.
All right, so far so good.
And maybe this column is something like an order ID
that we wanna use to kind of track each individual order.
In which case we could rename the column,
but to be honest, we don't need this ID or key column here.
In fact, it actually might just complicate
and confuse things down the road.
So, what I'm gonna do is
I'm gonna right click and delete this.
Now, what's interesting here is
I've basically, landed exactly where I started.
This is the table in its original kind of untouched form.
All the same columns that we've started with
and that we imported.
But one of the things that you'll notice here is that
I've actually got six different applied steps now
and what's happened is when we've removed the column
it didn't delete these other applied steps.
So, if I click back to the Add Index column applied step,
you can see that it's actually still here.
So, the implications of this are that
each time we refresh this dataset,
or we add more data to it, or like something changes,
Power BI is gonna run through
all of these different applied steps.
And that means adding the index column,
reordering the column, and then deleting it.
And that's just, honestly, a little bit silly and redundant,
and it isn't efficient at all.
So, what I'm gonna do here is I'm gonna

delete these last three applied steps
to remove that Add Index column work.
And now we're back to our original dataset
with our original three applied steps.
So again, this is just kind of
an important nuance to keep in mind,
and it's just a good demonstration
of how these applied steps are actually working.
Now, our last little demo here
I wanna create a conditional column
and it's right there above that index column step
and we get our dialogue box that opens here.
And so, first things first,
let's call this Quantity Type.
And then we have our different inputs here
for the conditional statement.
And like we saw in our slides,
we're gonna base this on our Order Quantity
and we wanna set this when it equals to one.
We want that output to equal single item.
Click Add Clause to add an additional case here.
So, if our Order Quantity equals one then single item,
otherwise or Else if, Order Quantity is greater than one,
we wanna return multiple items.
So, at this point, we're gonna be all set
for our conditional column,
but in cases where you have more complex logic,
you can continue to add new clauses here
to create additional Else if statements
for part of that conditional logic.
In this case, what we have here is a fully comprehensive
and mutually exclusive list of conditions.
We don't really have any other rows in our table here
with an Order Quantity of zero.
And I sure hope we don't have any rows
with a quantity of a negative value.
So that really leaves us with just two possible options.
You've either ordered one thing
or you've ordered more than one thing.

So, that said, as a best practice,
I always like to have an Else clause.
So, if either of these two conditions aren't met,
we're gonna say Else Other.
And again, this is just a great kind of catchall
in case something really strange or weird
or unanticipated happens within the dataset, we're covered.
We'll be able to flag that.
So, click OK.
And you can see here in the Quantity Type column
that we're getting our values
of multiple and single returned.
We can click the column header here
and we can see multiple item and single item.
One thing to notice here is that
you've got this kind of warning flag here
where it says, "List may be incomplete."
What's happening here is that when you see this flag
it's just Power BI telling you that,
hey, I only looked at a sample of rows within the column
in order to produce this list above.
And this is really just Power BI's way
of saving some memory and processing power.
But if you want Power BI to check
the entire contents of the column, click Load More.
And that kind of forces Power BI
to scan all of the values within the column
instead of just a preview.
All right, so this confirms
that we have just two Quantity Types, multiple and single.
So from here, we'll click OK.
And that's just about all we need to do here.
So, we'll head back to our Home tab.
We'll click Close & Apply,
and then we'll make sure to save this workbook.
So, we've got our sales data 2022
loaded into our model, and we'll keep moving on.



3:09 / 3:12

Instructor: I'd like to take a moment and talk about some best practices when it comes to creating calculated columns or adding new features within a data set. So as a best practice, table transformations and column calculations should ideally happen as close to the original data source as possible. Now, the reason behind this best practice is that when calculations happen at the source,

you're able to use the original data source engine like the SQL Server Engine or the MySQL Engine or Excel's workbook engine and not rely on Power BI's Engine to do these calculations. Basically, all of this boils down to optimizing Power BI reports for performance and speed. Here's a high level look at the different places where calculation should be made or added as part of a report development project. Ideally, new data features and columns are added as far upstream as possible, which means as close to the original data source as possible. And if that can't happen, then your next best option is to create calculated columns using the query editor, followed by the power BI front end and last in published reports or dashboards if that functionality exists. So at this point, you might be thinking that Power Query and Power BI's front end are all part of the same Power BI tool. Why is Power Query more efficient than the front end? And that's an awesome question. The reason is that Power BI's internal engine, called Vertipaq, creates something called the query plan when you press load and apply to load the data into Power BI's data model. And this query plan takes all of the raw source data transformation steps, any calculations that have been made, columns that have been added, et cetera, and it determines the best way to compress all of that data before loading it into Power BI's memory. Remember that lecture on storage and connection modes? When you're using import mode, the data is loaded into Power BI and the Veripaq engine needs to create this highly optimized plan for fast and reliable performance. So when new calculations, new columns and other data features are added using Power BI's front end they're not added to that same query plan

and can't take advantage of the compression methods.
So because of this, you might disproportionately
bloat your data model size.

Now, there is a lot of nuance involved here
and it's generally outside the scope of this course,
but just keep in mind that when possible,
add columns at the source, if you can't,
then try the query editor, if that doesn't work
then use the front end tools in Power BI.

One last call out here before we move on.
This best practice isn't a strict requirement or rule
but it's really something
that can significantly impact performance
for very large or complex data models.

And I totally understand that where you define calculation
often depends on several factors
like accessibility to data sources, complexity,
business requirements, your own ability
and level within Power BI,
so basically we'll be practicing different methods
to create columns using both the query editor,
like we've already done, and DAX,
which is found in Power BI's front end.



7:33 / 7:35

Instructor: Up next, I want to talk about grouping and aggregating data.

So within the transform tab of the Query Editor you'll see this Group By option on the left and this allows you to aggregate or roll up your data at different levels of granularity. So one of the common examples of this would be doing something like transforming

daily transactions into weekly or monthly
or rolling up transaction level data by store, by product,
brand or region, et cetera.

So it's taking a really deep, really detailed table
and rolling it up into a higher level summary.

So let's take a quick look at an example.

Here we've got order quantities by order date, product key,
and customer key.

And the important thing to note here
is that we have multiple orders for a given product key.

In this case the product number 214
has been ordered multiple times on multiple days
by multiple different customers.

And if you wanted to transform this table
into a summary of orders
or order quantities rolled up by unique product keys,
the Group By option is a great way to do that.

So when you click the Group By button
you'll see this dialogue box that looks something like this
that basically allows you to tell Power BI,
hey, I'd like to group this table by unique product keys.

And the operation that I'd like to evaluate
for those product keys is the sum
of the order quantity column.

In other words, you're taking all of these duplicate rows
with the same product key and multiple order quantities
and you're compressing them down to one.

When you do that compression or aggregation
how do we want to treat those order quantity values?

Now in this case, we're taking a simple sum
and we're gonna end up with a table like this,
two columns, one containing unique distinct product keys,
and the second containing the total quantity
or the sum of order quantity values
associated with each product key.

So to recap, what we've essentially done is transform
a daily transaction level table into a summary
of total quantity rolled up by product keys.

And the important thing to note here

as you may have noticed,
any fields that we aren't specifically calling out
in our Group By settings are lost.
They're not preserved in that final table
because we've eliminated that level of granularity.
Now, one more example, using the advanced option
and let's consider that same exact table
that we started with.

We're using the same exact Group By option
but this time we're looking at the advanced version.
And really the only difference here between the basic
and advanced is that advanced allows you to specify
multiple columns or additional columns to group by.
So this time, instead of just grouping by product key
we're grouping by product key and customer key.
And again, we're evaluating that same operation.
The sum of the order quantity values, just like before.
This time, what we end up with is a three column table
with product key, customer key and total quantity.
So to recap, we've again transformed
that daily transaction level table
now into a summary of total quantity
aggregated by both product key and customer key.
In other words, we're now looking at quantities
by each unique combination of those two fields.
And the best comparison that I can make to this,
it's really just like creating a pivot table
and pulling in the sum of order quantity
into your values and your product key
and customer fields into your row labels.
All right, so that's grouping and aggregating data
in a nutshell.

Let's jump over to Power BI
and we'll actually work through an example or two.
All right, so back in my Adventure Works report.
And instead of adding new data
we're gonna jump right back to the Query Editor
and we're gonna edit the sales data table.
All right, so if we select our sales data 2022 table

which contains daily sales records
broken down by product and customer keys, right?
Territory keys, right?
This is a great candidate for using Group By
and aggregation tools.
So just like in our demo
we're gonna select the product key here, all right?
And let's say that we want to group this entire table
which contains multiple product keys
and multiple instances of product keys, right?
And we can see that if I sort ascending, right
we've got all of these different product keys
that are repeating for different orders,
and we could scroll down here
might take a while if we scroll, but trust me, right,
there's a lot of different product keys
contained within this table.
So the idea here is that we want to take this table,
collapse these rows with multiple product keys
into essentially a summary table.
And just like the example on the slide
we want to evaluate the sum of order quantity
for each one of these product keys.
So from here, what we wanna do is head up
to the Transform tab, and then we're gonna click Group By.
And we're gonna start with our basic option here.
So we want to group by product key
and for our new column name
let's call this something like Total Quantity
and the operation that we want to perform here
we have a bunch of different options
for some different statistics tools
or aggregation functions.
So sum, average, median min, max, et cetera, right?
And we want to find the sum of our order quantity column.
So we'll press OK,
and just like our demo
this collapsed our data into a two column table
that's now aggregated by the quantity field

with the associated product keys, right?

And we have a unique list of product keys here which we can see again if we sort this column, right?

So instead of having multiple repeated values and rows for product 214.

We have a single row for 214, 215, 220, and so on.

Now let's do one more example here and I'm actually gonna remove these applied steps here. So we'll select our product key column again.

Come back up, we'll group by, and this time I wanna select the advanced button, right? So what this again allows us to do is add a grouping here and you can add any other sort of fields that you want to pull in from the table, right, by date, order number, customer key, territory key, you know, so on.

And here I'm going to select customer key because I wanna see the total sales for every combination of product and customer.

And again, we can call this total quantity.

The operation we'll do a sum, and we're gonna base that on the order quantity column.

We click OK.

And now our result changes.

We now have a three column table containing the unique combinations of product key, and we'll sort this, right of product key, customer key, and our total quantity, right?

So now you can see that we have multiple instances of product key because we've added another dimension here, our customer key.

So there you have it.

And once again, let's go back and we'll remove these applied steps for grouping and sorting.

And at this point, we actually don't need to save any of this because we don't wanna load any of these changes

into the data model.

We really haven't made any sort of significant changes.

So no real action is needed here.

All right, that's gonna wrap up your crash course
on grouping and aggregating data.

0:04 / 7:26

Instructor: Okay, so by this point, you should be pretty comfortable with the basic query editing tools, your date tools, and text tools, and number tools.

And for the most part, they've all been pretty straightforward, and pretty intuitive, but there are some pretty interesting, maybe not-so-intuitive tools that are available within the query editor.

And one thing that I wanna cover now is this idea of pivoting and unpivoting a table.

So the way I describe it, pivoting is just a fancy way to describe the process of turning distinct row values into columns, which is called pivoting, or turning columns into rows, which is called unpivoting.

Now it sounds pretty simple on the surface, but it's actually a little bit tricky to get the hang of, at least it was for me.

And I'm a visual learner, and what I'm about to show you is what finally caused it to click for me.

So imagine you've got this simple table, you've got Dates in the first column followed by Product Categories, then Region, and Quantity Sold.

If we were to select the Region and Quantity Sold columns, and then pivot this table, we would essentially take the region in each row and turn it into its own column.

So instead of having a row for North Region, we would have a column, and the same applies for Central and South Regions too. So what we've basically done here is

flip that table from a vertical format
up into a horizontal format.
We've turned rows into columns.
Now on the flip side,
if we started out with the table in that horizontal format,
and we selected the regions in the first row,
we could unpivot to transform it into that vertical form.
Now, in other words,
we're taking those regions which had been columns
and transform them into rows.
So my little mental tip here is that
I imagine that the table is kind of like on a hinge
in that upper left-hand corner.
Pivoting is like rotating it up from vertical to horizontal,
and unpivoting is like rotating down
from horizontal to vertical.
Now one thing to note here,
there's actually another tool
in the query editor called Transpose,
and that works in a really similar way,
but the difference is that
it doesn't recognize unique values.
So if you're dealing with a table
like the one we're looking at here
where we have duplicate dates and quantities,
pivoting and transposing would yield very different results.
So one way to think about that is that
Transpose is kind of clunky,
it's kind of like using brute force.
It just takes an entire table,
and flips it up on its side, right?
So you would have duplicates for each date, and category,
and so on, and so forth.
So I actually have a little demo file here
that I'm gonna show you,
because you really do have to see this happening
in real time in order to kind of understand
and grasp what's going on.
So let's jump over to Power BI,

and we'll show you what this looks like.
All right, so we're gonna go back up,
and we are gonna grab a new dataset,
and there is a dataset called
Product Category Sales Unpivot Demo.
And you can open this one to kind of follow along,
and play around with the dataset on your own
if you're so inclined.
Right, we get our preview,
and we'll add this in.
Again, for the sake of the demo,
we don't really need to update the name, or data types,
or anything like that.
If there's any errors here, you could address those,
but everything should be good to go.
All right, so you can see here we're actually starting off
with this table pivoted.
And let's say that we were given this table
by a team of sales reps that had been collecting sales data
for each of their regions by product and date,
and they combined all of their work
into a single data set, right?
And they give it to you, and they say,
"Hey, we need some help with some data visualization here.
And we really wanna be able to filter all of our sales
by date, and product, and region."
And the way that this data set's configured,
it doesn't make that very easy to add a Region filter.
So, as an analyst,
as someone who needs to take this data and interpret it,
and analyze it, and explore it,
this type of table format can lead
to a bunch of different headaches.
And really what we're looking for here
is a rectangular table with each dimension, or metric,
as its own column,
and each observation as a row.
So, ideally, what we're looking for here is
to transform this whole table into a four-column table

that contains date, product category, a column for region,
and then another column for my sales quantity.
Once we get it in that format,
that would give me a form
that I could then plug into my data model,
and analyze in any way that I choose.
So let's take a look at
some of these pivoting and unpivoting tools
to see if they can help us.
All right, so because we're starting in this pivoted mode,
what we wanna do is we wanna unpivot these columns, right?
So I'm gonna select my different regions here,
and we'll come up to Transform,
and you can see we have Unpivot.
I've got a few options here:
Unpivot Columns, Unpivot Other Columns,
Unpivot Only Selected Columns.
And because we've selected the columns we wanna unpivot,
we'll click Unpivot Columns, right?
And what this does is it creates an Attribute column,
and then a Value column, right?
So from here, we could name this Region, right?
And this could be, you know, Quantity Sold.
So very quickly, we've been able to get our data set
in a format that lends itself a lot better
to being analyzed.
Now we could go back to the pivoted version.
All right, so we select our two columns,
we'll come up here to Pivot,
and from our dialog,
use the names in the column Region to create new columns.
Our Value column is our Quantity Sold.
So we're good there.
We'll click Okay, right?
And then we're back to that pivoted version of the table.
So, again, pretty straightforward demo here
when you've got a table that contains columns like this,
and you need to get it into a little bit better of a format.
What we can do here is we can delete

these last applied steps here
to return the table back to its original format, right?
And the last thing that I wanna show you here is that
the Transpose option that we talked about
in the slides here, right?
Again, like that's this brute force option.
You take a look at our rows here, right?
We've got four different rows for July 1st.
We've got four different rows for July 2nd.
And what happens is it retains everything,
it doesn't group it,
so if we click Transpose,
you can see here that we've got four columns
for July 1st, right?
For Bike, Components, Clothing, Accessories, right?
If we scroll over, we've got four rows for the second,
again, Bike, Component, Clothing, Accessories.
So again, using Transpose, in this case, doesn't make sense,
because, again, it's gonna split these all
into their own columns.
And, again, that's not the desired state
that we're looking for.
All right, so there you have it.
That's your demo on pivoting and unpivoting a dataset.

6:41 / 6:44

Instructor: All right, up next we have Merging Queries.
So you'll see the Merge Query options here
in the Home tab of the Query Editor.
And when you click on that button,
it'll open up a dialogue box like this,
where you can select two tables with matching columns
in order to create a merged table.
And basically, this works just like a lookup in Excel.
You're joining information from two different tables
based on some common column or key.
So in this example that we're looking at here,

we're merging a table called Sales Data
with another table called Product Lookup,
both of which contain a matching product key column.
So essentially, what we'll be doing here
is grabbing information or attributes
about those product keys from the Product Lookup table
and pulling them in or embedding them into the Sales table
by matching them to the associated product keys.
So my trick for remembering this
is that merging adds columns to an existing table.
So you're not adding any rows,
you're pulling in more information
and making your table wider by adding columns.
So very important point here,
just because you can merge tables
and because it's an interesting tool to use,
that doesn't mean that you should.
In fact, it's generally better to keep tables separate
and define relationships between them within the data model,
and again, more on that later,
but keep that idea in the back of your head,
that you can merge queries like this
but it's always not the most efficient solution.
So why don't we go and jump into the Query Editor,
and I'll show you
what one of these merges actually looks like.
Okay, so I'm in the Query Editor
of my AdventureWorks Report,
and to demo the idea of merging,
let's say that we wanna look at our sales data,
which has our order dates and order numbers,
our product key, customer key, territory key,
order line items, quantities, right?
We've got all of these attributes related to our sales.
And let's say we wanna pull in a little information
about these products right here into the Sales table
without having to go over into the Product Lookup,
find the associated product key,
and then look at the information here, right?

Let's just say we want all of that information accessible in one place.

So this is a opportunity that we could use a tool like Merge.

So I'm gonna start with the Sales Data here, and from my Home tab, and all the way over on the right here, you're gonna see options for Merge and Append, and we'll talk about Append next.

But with Merge, I can do it in a couple of different ways. I can either Merge Queries or Merge Queries as New. And if I just simply use the Merge Queries option, it will preserve the exact tables that I have in place now and it will just add the merged columns into my existing Sales Data 2022 table.

If I choose the second option, it's gonna create a brand new merged table that contains the merged results while maintaining my original component tables as well. So I really don't want or need this option at this point so we're gonna click on Merge Queries.

So what we have here is we have our first table, right? We had selected the Sales Data 2022 table.

So this is the first table that shows up in our Merge Queries option, right?

We can see all of the columns contained within that table. And then below that we have this dropdown menu, and this allows us to access all of the other queries that have been added into the Query Editor.

And again, we wanna grab detail from the Product Lookup table, right?

And so then again here we get this preview of all of the columns contained within our Product Lookup table.

And the only columns that these two tables have in common is Product Key.

So if I select Product Key and Product Key, Power BI goes through, you can see down at the bottom here, it says, "The selection matches 29,481 of 29,481 rows

from the first table."

So it's an exact match from product to product.

The other thing to pay attention to here

but not something that we necessarily have to worry about
is the join type, right?

So by default, this join is a left outer.

And what this is basically saying

is take all of the product keys

that exist on my first table,

this Sales Data 2022 table,

and grab any rows that you find a match for

in my second table, this Product Lookup table.

And for our purposes, this is gonna do the trick.

So everything looks good here, we'll click OK.

And so you see what happens here,

we've added a new column

but the column looks kind of funny.

It says, "Table," and it's formatted with these links.

All right, so from here,

we actually have one extra step that we need to take,

and it's to click this Double Arrow button here

to expand the table and all of the contents within it.

Right, so by default, all of the columns

from the table that we wanna join are selected,

and if we were to press OK here,

this would add all of these columns as part of the merge.

And honestly, that's not really what we want.

So we wanna go through

and select the columns that we actually wanna see.

So let's start off, we'll select Product Key.

This will give us a way to kinda compare results.

And then let's say we want Product Name and Product Color.

Our last checkbox here,

Use Original Column Names as the Prefix,

if I leave this selected,

it's gonna create a prefix for every column name

with the name of the product table, right?

And we'll see what that looks like in a minute.

I'll just leave this selected for the sake of this example.

So we click OK.

And now we've expanded the Product Lookup table.

And now we've got this new information
that didn't exist in the sales table
up until this point, right?

We've got Product Name and Product Color detail,
and it's populated for every single row of the table,
just like it would if we were using an Excel Lookup, right?

So we can scroll over to the left here, right?

And you can see we've got Product Key 529, 529, 214.

Scroll back over.

Right, we've got 529, 529, 214, right?

So we can see that we've got all of this new data added in
for every record of the table.

And, like I hinted at earlier,
just because we can do things like this
and merge tables together
doesn't mean that we always should.

In fact, merging information in the way that we just did
actually creates a lot of duplicative and redundant data,
which can be avoided entirely by using table relationships.

Again, we're getting there soon.

We'll cover relationships in the next section.

But for now, let's go ahead and we'll delete that merge.

And now we are back to where we started.

So there's a quick demo for you on how to merge queries.

Instructor: Up next we've got appending queries, and you'll find the appending queries tool right under Merge in the Home tab of the query editor. When you click that button, you'll see this really simple dialogue box where you can choose to append either two tables or three or more together. Now, both merging and appending are similar in that they're adding information to an existing table, but they're different in that merging is adding columns and appending is stacking rows. So basically, appending queries allows you to combine or stack two tables, assuming that they share the exact same column structure and data types. So that's super important to keep in mind. Same structure and same data types. So in this case that we're looking at here, we're appending two tables together. There are two different AdventureWorks sales tables, one from 2020 and one from 2021, and it's a valid append since they both share identical table structures. So just like we've talked about merging makes tables wider by adding more columns, appending makes tables taller by adding more rows. So a pro tip here that I wanna share with you and we'll actually dive into this with the demo in the next lecture is to use the folder option from the Get Data menu. And what this lets you do is actually point Power BI to a folder containing as many files as you choose, which allows you to append all of them at once, assuming that they share that same structure. And the beauty of it is that over time if you add new files or remove files, simply refreshing the query, refreshing that connection will automatically append any new data that's shown up within the folder.

So it's a really powerful option that we'll dive into within the next video. But for now, let's cover the basic append options and we'll shift gears into Power bi so we can take a look. Alright, so we're inside the Query Editor and I'm in my Sales Data 2022 table. And we've got a bunch of data sources in here. But the problem here is that none of these data sources share the exact same structure or data type. So what we need to do is we actually need to go and get some new data. Again, we're gonna do this right from our New Source tools, and we're gonna go and we're actually gonna bring in our other two sales data sets. So let's connect to 2021. All right, preview looks good. All right, I'm gonna quickly update my table name again, and then our column headers, data types, everything looks good here. All right, let's go back. We'll get our 2020 data set. And we'll add this into the query editor. And following our very similar repeatable process, we've got our data preview. Again, everything looks good here. Same steps. We'll update our table name and then I'll just confirm again that our data types, column headers, everything looks good. All right, so now we've got these three separate sales data files, right? For 2020, 2021, and 2022. And they are all consistent in terms of the number of columns, except for our 2022 data set, right? This is where we had added in our quantity type column when we were practicing our conditional column tools. So one option would be to go ahead and continue to append these together.

You'll see some null values
for the quantity types in the other two tables.
And you could pretty easily redefine the conditional column
to the full append version.
In this case, we're just gonna go ahead and remove it.
So we come over to this column,
we can actually delete our last applied steps.
And now we're back to the same exact structure
across all three of these tables.
So with one of them selected,
it doesn't really matter which one,
I've got 2022 selected here,
from the Home tab, we're gonna go to the Append Queries.
Again, right below merge.
And again, we've got those same options
that we saw in the Merge Queries.
We can append queries or append queries as new.
And for this example, I wanna append queries as new.
I wanna create a new table that's based
on these other three queries.
All right, so we've got our dialogue that pops up
and we actually have two or more tables,
so we're gonna collect three or more tables.
You can see here that we already are showing
our sales data 2022 as part of the append,
'cause that's what's selected,
and then we have all of our other available tables here.
And if we scroll down, we can hold down Shift
and click 2021 and 2020, and then click Add,
and now we've got all three of these tables
that we are gonna append together.
Click OK.
So here we go, we've created this new table.
Its name is just Appended1 by default,
so let's update that to something
a little bit more readable.
So we'll call this Sales Data 2020 through 2022.
Again, just to be clear.
And now we can look through the column structure here.

We can see that our columns are exactly the same.
We select our order date drop-down and view more.
We've got all of our 2020.
Scroll all the way down here.
You can see all the way down to June 30th, 2022.
All right, and then probably in the middle here,
we've got all of our 2021 data.
So everything looks good there.
We've successfully appended all of our sales data together.
So everything might seem great,
but one downside to this approach
is now that I've got this table with all of the data in it,
let's say I wanna get rid of these individual tables here,
if I right-click and select Delete,
I get a message from the query editor saying like,
"Hey, we can't delete this table,
because it's being referenced by another query."
And Power BI won't let me actually delete this
because it needs this table in order to populate
this table that's appended.
So the problem here is that our new table,
our Sales Data 2020 through 2022,
is dependent on three other tables.
So we kinda have to keep all four of 'em here,
which it's okay, but honestly it's not the greatest.
We've got duplicate redundant queries in the editor,
and while it's probably not really
gonna create any big problems,
it's just messy and not great form.
So one way around that, which I recommend very much,
is to move these files into a folder
and then connect to the folder itself.
So stay tuned, because in the next video,
I'm gonna walk you through exactly how to do that.

7:33 / 7:36

Instructor: All right, so we just walked through how to stack some tables together using the Query Editor append tools and we took these three individual sales data files from 2022, '21, and '20 and combined them into a new appended data set, right, where we stacked all three of them on top of each other. And that worked pretty well. But remember the downside, this new table we created is now dependent on three individual component tables. So we can't delete any of these individual tables or if we need to add in new years worth of data, like, we'll have to reconfigure the append to make that work. So, again, it's just, it works decently but it doesn't scale very well. So what we're gonna do here is we're gonna actually delete these four sales queries and then we're gonna use Power BI and connect to a folder instead, right? So first things first, we can delete this append query and then once that's removed, we can actually start removing our individual sales data files, right? Because now, they're not referencing any other queries. And delete 2021. And then our last file here for 2022, we'll delete that. All right, and we're good to go here. So now that we've done that, we're left with our six kind of main lookup tables that we're gonna be using to build out our data model. And from here, what we want to do is we actually want to go in and connect to a new source, right? That's gonna be that folder. And I quickly want to show you

what's contained within that folder.
So I'm gonna navigate to that folder on my desktop.
All right, so this is the folder
where we've got all of the AdventureWorks data
and I have this sales data folder here.
I double click into this,
you'll see that we've got our three sales tables
for 2022, '21, and '20 within here.
And this is what we're gonna be connecting to
from the Power BI folder data source.
So we'll jump back to the Query Editor
and we'll come up here to a new source,
and I'm gonna click more for more options.
All right, and from the Get Data window,
you actually see we've got our folder option right here.
We can click connect
and the Query Editor prompts us with the folder path.
Click browse to browse your file structure.
Right, desktop, Adventure Works Raw Data,
and then our sales data, right?
So this is the file path that we want.
Click okay.
And again, this is gonna follow that same process
where it's gonna bring up a data preview window.
We'll be able to check out what that data looks like.
Now, what's interesting here is this preview window
looks a little bit different, right?
It's got this content column that says binary.
It's got file names, and extensions,
and date accessed, and modified.
It's got the folder path string.
So rather than actually previewing the data,
the actual rows and the columns,
what we're actually previewing are the files that exist
within the folder path, right?
So it's a little bit different here
but a similar process, right?
And we've got two options as well.
We can combine and transform

or we can just transform the data.
Either one of these options
is gonna kind of put us into the same spot,
but I'm gonna click transform data
so we can walk through each of these steps.
So here we go, we pressed edit,
we're back in our Query Editor,
and we see that similar kind of view
that we got in preview, right?
We got content and the name,
our file extension, date modified,
created our folder path, and all that kind of stuff.
So again, we're still not looking at data points here.
We're still looking at the attributes of the files
that exist within that folder that we've linked to.
The key here is the first column.
And this column is kind of similar to the column
that was created when we merged tables
where we had to take one extra step
to actually split out the columns from the table itself.
It's the same idea here.
So you can see when I hover, it says, "Combine Files."
And when we press this,
it runs through a different append process.
So once Power BI finishes that behind the scenes process,
we're brought to this new window
with this combine files header.
And this looks like the data preview that we were seeing
when we brought in our individual sales data tables, right?
Our order and stock dates, order number,
product key, customer key, you know,
order line items, order quantity, right?
So this all looks very familiar.
The other thing that's interesting here too
is if you click on the sample file,
the default is the first file
that you've connected to, right?
So 2020.
But you can actually click through and view previews

based on the other data tables
that are contained within that folder, right?
So this is just another great way to kind of go in
and check to make sure that column structures
and all of that stuff is the exact same.
All right, so everything looks good here,
kind of checks out.
We've got 2021 for our order date.
From here, we're gonna click okay.
All right, we've got a whole bunch
of different applied steps here
that were automatically generated.
If we check out our order date column here, we can load more
and we can actually see, so we've got 2020, 2021, 2022,
all the way through June, right?
So everything looks good there.
Our column structure is all the exact same
with the exception of this new source name column.
And what this column is doing is it just tells us
which file from within the folder that the data came from.
So it's potentially a helpful attribute
if you do want to keep it.
In this case, we really don't need it
so let's go ahead and we can remove this column
from our sales data table.
And there you have it.
We have our appended table
containing all of our sales data in one place.
And as you can probably see here,
it gave us an automatic table name
that's based on the folder, right?
So our folder name was Sales Data, right?
So we could update this if we wanted to.
Honestly, I think that this is a great name for it
so we're gonna leave it as is.
One last thing to note here,
over on the left hand side of the screen
where the queries are,
is we've got a whole bunch of other detail

and information that was automatically generated by the Query Editor.

And basically, what this is is this is all of the behind the scenes stuff that Power BI needed to do in order to append those three files together from that folder.

So typically, what I'll do here is I'll just collapse these queries, right?

And just kind of hide them.

And that way, I can kind of stay focused on my other queries that I have here.

We are ending up with that same end result as appending those three individual tables together, except now, we've got one nice clean table without all of those extra tables and dependencies.

And even more importantly, if we ever needed to pull in additional years of sales data, we could simply drop those CSV files right into that folder, press refresh right here inside the Query Editor and all of that data would be automatically pulled in.

So that's just about do it here for this demo on connecting to data that's located in a folder.

Let's come up here, we'll click close and apply.

And Power BI is working on updating all of those changes that we made within the Query Editor.

All right, so once this wraps up, we should see that sales data 2022 table disappear and just be updated with the sales data table.

All right, so it looks like we're good to go here.

And that's gonna wrap up your pro tip on connecting to data that's located in a folder.

Narrator: Okay, so up until this point, we've talked about a number of the most important data transformation tools in the query editor.

Up next, I wanna cover some important settings and options. And the first topic here is data source settings.

So in the home tab of the query editor,

you see this option for data source settings.

And that basically allows you to view and manage all of the individual connections in your Power BI file.

So in this example that we're looking at here, we've got connections to a number of individual CSV files as well as a folder down there at the bottom of the list.

Now, the critical concept to understand about the data source settings is that when you make a connection to a local file, a file that's located on your computer.

Power BI will basically take a snapshot of that exact file path and the exact file name.

And what that means is that if anything changes about your file, if you rename it, if you move it, update permissions, a path location, anything, that path won't automatically update within Power BI.

So, what you need to do is go into these data source settings, click change source and then browse to the revised version of that file.

So, this is probably one of the most common issues that I see in Power BI, where people inadvertently rename a file or rearrange things on their desktop and accidentally break these file paths.

But the good news is that it's very easy to fix as soon as you understand how these data source settings work.

So, let me show you what I mean.

For this video, you can either continue to just follow along here with me and watch, because we're not actually going to be adding any new data or revising anything to what we have.

I just wanna show you a demo of what happens when a file path does break and you need to update it. All right, so here I'm gonna head to the query editor and from here to check out our data source settings, right? We're from the home tab, data source settings and this dialogue will open up that shows us all of the different connections within the workbook, right? So, we've got our Maven Fuzzy Factory SQL Connection, we've got a bunch of local CSV file connections, a folder. We've got our Wikipedia web link, right? So, we're seeing everything here and everything's looking good. It's running smoothly. But what would happen if I accidentally changed one of these file names? So, let me navigate to my actual file folder on my desktop. All right, here we are. And let's say for example, I'm gonna update the AdventureWorks product lookup name and we'll call this _2, right? So, we'll rename that and now I'm gonna head back to the query editor. And if I click on my product lookup table and I refresh this data, we get an error here, right? Could not find the file, right? Because what Power BI is looking for is this AdventureWorks product lookup table, but it's now named Lookup_2, right? So there's a couple of different ways that we can go about actually fixing this. If we go back up to our data source settings, we can come down to our product lookup table and we're gonna change the source, right? So what this does is it allows us to browse for a new path and we can come back here, we'll point it at our new lookup table, click Open, OK. And then we can close. And now when we refresh the data, everything looks good here.

These applied steps all work, right?
And we're left with a usable product lookup table again.
Now we're back to usual, sources populated properly.
All of the applied steps are maintained, like we just saw,
we haven't lost any information.
Pretty simple fix.
We could just leave things the way they are.
Or I can kind of show you that same process
in reverse, right?
So, what I'm gonna do is I'm gonna head back
and I'm gonna change the file back
to its original name, right?
Back here and we'll remove the two.
We'll head back to the query editor, right?
And I'm actually gonna refresh this again,
so we can get that error to show up, right?
So now, Power BI is looking for two
and we just updated the file name.
So, remember how I said there's a couple
of different ways that you can update this.
We've already seen the process
by going through the data source settings.
What you can actually do here
is from your source applied step, click
on this gear icon and this brings you back
to that same exact place where you can now browse
for the correct file path or the file location, right?
So, we'll click on our product lookup, same process, Open,
click OK, and that's been updated again.
If we go down to our last applied step,
everything looks great.
It's been updated, no data loss
and we've successfully reverted
that file name back to its original name.
Hopefully, that was a helpful demo
on how to address some of those data source errors
using the data source setting options.

Narrator: Okay, so up until this point, we've talked about a number of the most important data transformation tools in the query editor.

Up next, I wanna cover some important settings and options. And the first topic here is data source settings.

So in the home tab of the query editor, you see this option for data source settings. And that basically allows you to view and manage all of the individual connections in your Power BI file. So in this example that we're looking at here, we've got connections to a number of individual CSV files as well as a folder down there at the bottom of the list.

Now, the critical concept to understand about the data source settings is that when you make a connection to a local file, a file that's located on your computer. Power BI will basically take a snapshot of that exact file path and the exact file name. And what that means is that if anything changes about your file, if you rename it, if you move it, update permissions, a path location, anything, that path won't automatically update within Power BI.

So, what you need to do is go into these data source settings, click change source and then browse to the revised version of that file. So, this is probably one of the most common issues that I see in Power BI, where people inadvertently rename a file or rearrange things on their desktop and accidentally break these file paths. But the good news is that it's very easy to fix as soon as you understand how these data source settings work.

So, let me show you what I mean. For this video, you can either continue to just follow along here with me and watch, because we're not actually going to be adding any new data or revising anything to what we have. I just wanna show you a demo of what happens

when a file path does break and you need to update it.
All right, so here I'm gonna head to the query editor
and from here to check out our data source settings, right?
We're from the home tab, data source settings
and this dialogue will open up that shows us all
of the different connections within the workbook, right?
So, we've got our Maven Fuzzy Factory SQL Connection,
we've got a bunch of local CSV file connections, a folder.
We've got our Wikipedia web link, right?
So, we're seeing everything here
and everything's looking good.
It's running smoothly.
But what would happen if I accidentally changed one
of these file names?
So, let me navigate to my actual file folder on my desktop.
All right, here we are.
And let's say for example, I'm gonna update
the AdventureWorks product lookup name
and we'll call this _2, right?
So, we'll rename that and now I'm gonna head back
to the query editor.
And if I click on my product lookup table
and I refresh this data, we get an error here, right?
Could not find the file, right?
Because what Power BI is looking for
is this AdventureWorks product lookup table,
but it's now named Lookup_2, right?
So there's a couple of different ways
that we can go about actually fixing this.
If we go back up to our data source settings,
we can come down to our product lookup table
and we're gonna change the source, right?
So what this does is it allows us to browse
for a new path and we can come back here,
we'll point it at our new lookup table, click Open, OK.
And then we can close.
And now when we refresh the data,
everything looks good here.
These applied steps all work, right?

And we're left with a usable product lookup table again.
Now we're back to usual, sources populated properly.
All of the applied steps are maintained, like we just saw,
we haven't lost any information.

Pretty simple fix.

We could just leave things the way they are.
Or I can kind of show you that same process
in reverse, right?

So, what I'm gonna do is I'm gonna head back
and I'm gonna change the file back
to its original name, right?

Back here and we'll remove the two.

We'll head back to the query editor, right?

And I'm actually gonna refresh this again,
so we can get that error to show up, right?

So now, Power BI is looking for two
and we just updated the file name.

So, remember how I said there's a couple
of different ways that you can update this.

We've already seen the process
by going through the data source settings.

What you can actually do here
is from your source applied step, click
on this gear icon and this brings you back
to that same exact place where you can now browse
for the correct file path or the file location, right?

So, we'll click on our product lookup, same process, Open,
click OK, and that's been updated again.

If we go down to our last applied step,
everything looks great.

It's been updated, no data loss
and we've successfully reverted
that file name back to its original name.

Hopefully, that was a helpful demo
on how to address some of those data source errors
using the data source setting options.

